Gymnasium Nepomucenum Coesfeld
Projektkurs Informatik/Physik

Computer im Eigenbau

Gustav Hisker

9. Juni 2024

Inhaltsverzeichnis

1 Einleitung **3**

2 Grundlegender Aufbau des Computers **4**
2.1 Das Speicherwerk 4
2.2 Das Steuerwerk 5
2.3 Das Rechenwerk 5
2.4 Die Ein- und Ausgabe 5
2.5 Die Busse 5

3 Generelles **6**
3.1 Die Komponenten 6
3.2 Busse und die Tristate-Logik 6
3.3 Logisim 7

4 Das Speicherwerk **8**
4.1 Die Speicherzelle 8
4.1.1 SR-Flipflop 8
4.1.2 Pegelgesteuerter D-Flipflop 9
4.1.3 Flankengesteuerter D-Flipflop 9
4.1.4 Flankengesteuerter D-Flipflop mit Aktivierung 10
4.2 Verknüpfung mehrerer Speicherzellen 11
4.2.1 Register 11
4.2.2 Addressierbarkeit mehrerer Register 12
4.3 Verkabelung 14

5 Das Rechenwerk **18**
5.1 Dem Computer Zahlen erklären 18
5.2 Addieren 18
5.3 Subtrahieren 21
5.4 Akkumulator 24

6 Das Steuerwerk **29**
6.1 Funktionen des Steuerwerks 29
6.2 Befehle 29
6.3 Das Verbinden der Komponenten 30
6.3.1 Die Kabel des Speicherwerks 30
6.3.2 Die Kabel des Rechenwerks 30
6.3.3 Alle Steuersignale 30
6.4 Dekodieren der Befehle 31

6.5 Implementation der Befehle . . . 31
6.5.1 Lade-Befehl . . . 32
6.5.2 Speicher-Befehl . . . 32
6.5.3 Addiere-Befehl . . . 32
6.5.4 Subtrahiere-Befehl . . . 32
6.6 Befehle aus dem Speicher laden . . . 33
6.6.1 Register im Steuerwerk . . . 35
6.6.2 Einbau in das bestehende Steuerwerk . . . 38
6.6.3 Sprungbefehle . . . 41

7 Beispiele für Programme **47**
7.1 Verdopplung . . . 47
7.2 Multiplizieren . . . 48
7.3 Zeiger . . . 49
7.4 Die Fibonacci-Folge . . . 50
7.5 Euklidischer Algorithmus . . . 51

8 Fazit **54**

9 Legende **56**

Kapitel 1

Einleitung

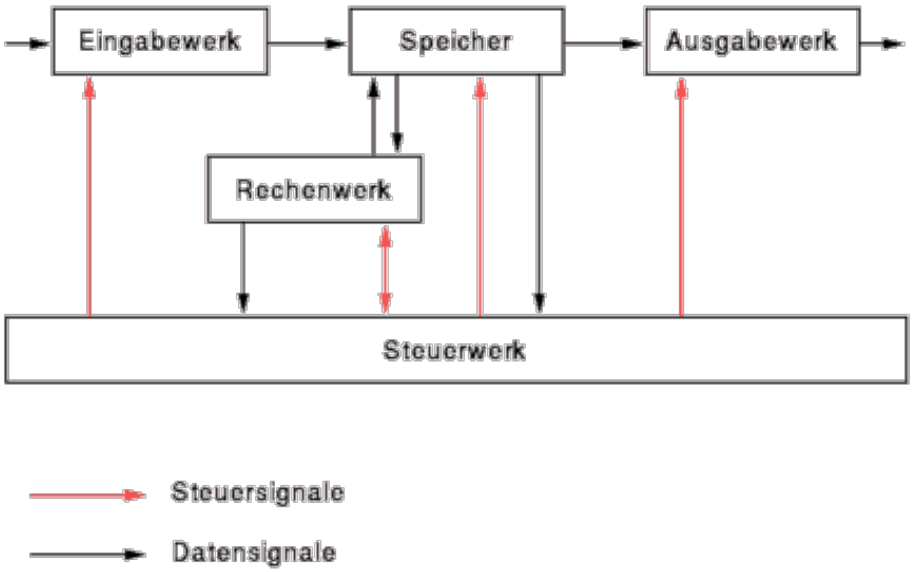
Computer sind aus unserem heutigen Alltag kaum noch wegzudenken, sei es in Form von Smartphones, Laptops oder auch nur Ticketautomaten. Doch es ist fast niemandem bekannt, wie sie funktionieren. Um dies genauer herauszufinden, habe ich mir das Ziel gesetzt, einen Computer selber zu bauen, und dies auf einer sehr grundlegenden Ebene, den Logik-Gattern. Damit möchte ich einen möglichst tiefen Einblick in die Funktionsweise eines Computers bekommen.
Ich habe mich bereits mit dem Aufbau von Computern befasst und kenne auch schon deren grundlegende theoretische Prinzipien, wie die von-Neumann-Architektur. Zudem habe ich mich auch bereits mit Schaltungen von Logik-Gattern befasst, allerdings noch nie in der Tiefe, in der ich es mit dieser Projektarbeit tue.
Ein sehr zufriedenstellendes Ziel wäre für mich das Ausführen von Programmen auf dem selbst gebauten Computer als ein Beweis seiner Funktionalität.

In folgender Projektarbeit wird das Ziel verfolgt, einen primitiven Computer auf Basis von einfachen logischen Basiskomponenten zu bauen. Mit Computer ist hier eine sehr einfache Form gemeint. Dieser soll in der Lage sein, Daten kurzfristig zu speichern und sie wieder aufzurufen sowie einfache arithmetische Operationen durchzuführen, all diese Befehle auch aus dem Speicher heraus aufrufen zu können und somit sogar kleinere Programme auszuführen. Die praktische Umsetzung dieser Ziele geschieht mangels Finanzierung nur in der Simulation.

Kapitel 2

Grundlegender Aufbau des Computers

Für meinen Computer benutze ich die von-Neumann-Architektur. Dieses Konzept teilt den Computer grundsätzlich in fünf Teile ein, die miteinander über sogenannte Busse kommunizieren[1]:



https://tams.informatik.uni-hamburg.de/applets/baukasten/DA/Bilder/von_neumann_rechner_50.gif (Letzter Aufruf: 15.5.2024)

2.1 Das Speicherwerk

Die Aufgabe des Speicherwerkes ist es, wie der Name schon sagt, Daten zu speichern. Dies wird durch mehrere Speicherregister einer einheitlichen Größe gewährleistet, von denen jede einzelne über eine gewisse Adresse ansprechbar ist.

[1]Kelling, "Aufbau, Konzepte und Funktionsweisen von Computersystemen".

2.2 Das Steuerwerk

Das Steuerwerk ist die zentrale Verwaltungsstelle im Computer. Es kümmert sich um die Ausführung der Befehle und sendet die nötigen Steuersignale an die verschiedenen Komponenten, empfängt Datensignale und richtet nach diesen seine Operationen aus. An das Speicherwerk sendet es zum Beispiel die Adresse, die es auslesen will und weist es an, die Daten auszugeben und empfängt diese Daten oder sendet die Adresse und weißt es an, die Daten, die es dem Rechenwerk angewiesen hat, zu senden und davor zu berechnen, dort zu speichern.

2.3 Das Rechenwerk

Das Rechenwerk führt die arithmetischen Operationen durch. Das kann je nach Größe erstmal nur Addition und Subtraktion beinhalten, aber sogar über Multiplikation und Division noch hinausgehen. Außerdem kann es auch validieren, ob eine Zahl 0 ist oder kleiner oder größer. Diese Wahrheitswerte können dann in das Steuerwerk einfließen und ihre Operationen beeinflussen.

2.4 Die Ein- und Ausgabe

Die Ein- und Ausgabemodule eines Computers, sind Kanäle, über die der Computer Daten empfangen kann – die Eingabe z.B. Maus, Tastatur, ... – beziehungsweise Daten aussenden kann – die Ausgabe, z.B. Grafikkarte, die dann die verarbeiteten Daten an einen oder mehrere Bildschirme schickt.
Für meine einfache Version eines Computers habe ich auf diesen Teil verzichtet, der Computer wird nur intern arbeiten.

2.5 Die Busse

Um einen möglichst übersichtlichen Aufbau zu kreieren und möglichst wenig Kabel zu benutzen, verlegt man Busse. Das sind Kabelstränge, die sich durch mehrere Komponenten ziehen. Auf ihnen werden zum Beispiel die Ergebnisse des Rechenwerkes an das Steuerwerk übermittelt. Es ist so Kabel sparend und übersichtlich, da auf einem Bus mehrere Komponenten ihre Daten senden und empfangen können. So könnte auf dem im Beispiel genannten Bus auch das Speicherwerk Daten zum Speichern empfangen bzw. welche senden, die es aus dem Speicher ausliest. In der hier gebauten einfachen Version eines Computers gibt es nur einen Bus.

Kapitel 3

Generelles

3.1 Die Komponenten

Die grundlegenden Komponenten, die ich plane zu nutzen, sind nur die Logik-Gatter mit den folgenden Operationen:

1. UND / AND
2. ODER / OR
3. NICHT / NOT
4. NICHT-UND / NAND
5. NICHT-ODER / NOR
6. EXKLUSIVES-ODER / XOR
7. TRISTATE-PUFFER / -BUFFER

Die genaue Funktion all dieser Komponenten ist in der Legende nachzulesen.

3.2 Busse und die Tristate-Logik

Die meisten handelsüblichen Logik-Gatter (z.B. in Form von IC-Chips) interpretieren eine Verbindung zum positiven Pol der Spannungsquelle als 1 und eine Verbindung zum Minuspol als 0. Das führt allerdings zu Problemen, wenn man mit einem Bussystem arbeitet. Wenn nämlich nun mehrere Komponenten am selben Bus angeschlossen sind und sie nicht denselben Zustand senden, ist an den Bus an einer Stelle der Plus- und an anderer Stelle der Minuspol angeschlossen. Infolgedessen tritt ein Kurzschluss auf, da nun der Strom einfach zwischen diesen Verbindungen über den Bus fließen kann.
Um dies zu lösen, benutzt man sogenannte Tristate-Puffer. Diese erlauben neben dem Zustand von 1 und 0 entsprechend einer Verbindung zum Plus- beziehungsweise Minuspol auch einen neutralen Zustand, also keine Verbindung zu einem der beiden Pole. Dies wird beim Tristate-Puffer erreicht, indem man zwei Eingänge benutzt: einen für das Signal, welches man zum Beispiel auf den Bus leiten will, und eins, das die Ausgabe dieses Signals aktiviert. Ist dieses Aktivierungssignal ausgeschaltet, so wird der eben beschriebene neutrale Zustand ausgegeben.

3.3 Logisim

Das Simulationsprogramm, welches ich in diesem Projekt verwende, ist Logisim-evolution[1]. Das ist ein kostenloses Open-Source-Programm, welches frei zum Downloaden zugänglich ist unter `https://github.com/logisim-evolution/logisim-evolution`. Mit diesem Programm kann ich die Schaltkreise bauen und sie auch simulieren. Alle in der Projektarbeit gezeigten Bilder der Schaltkreise sind dieser Software entnommen.

[1] Git-Hub, *Logisim-evolution*.

Kapitel 4

Das Speicherwerk

Um zu erklären, wie ein Speicherwerk funktioniert, werde ich im Folgenden erklären, wie es überhaupt Daten speichert, diese gut abrufbar speichert sowie auch größere Mengen an Daten speichert.

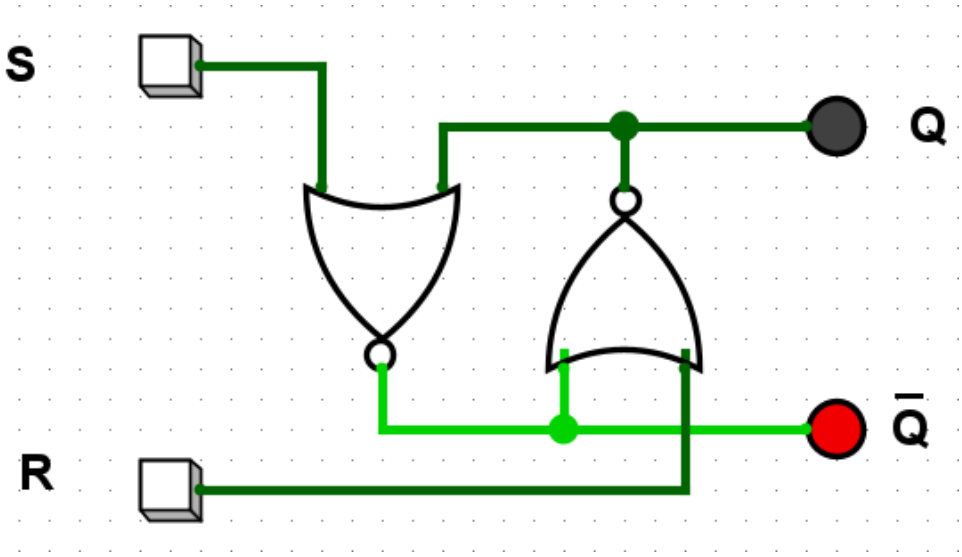
4.1 Die Speicherzelle

Dieser kleine Teil des Speichers kann genau einen Bit (an oder aus bzw. 1 oder 0) speichern. Der Speicher ist am Ende nicht mehr als eine wohldurchdachte Verdrahtung vieler (in diesem Fall 2048) solcher Zellen.

4.1.1 SR-Flipflop

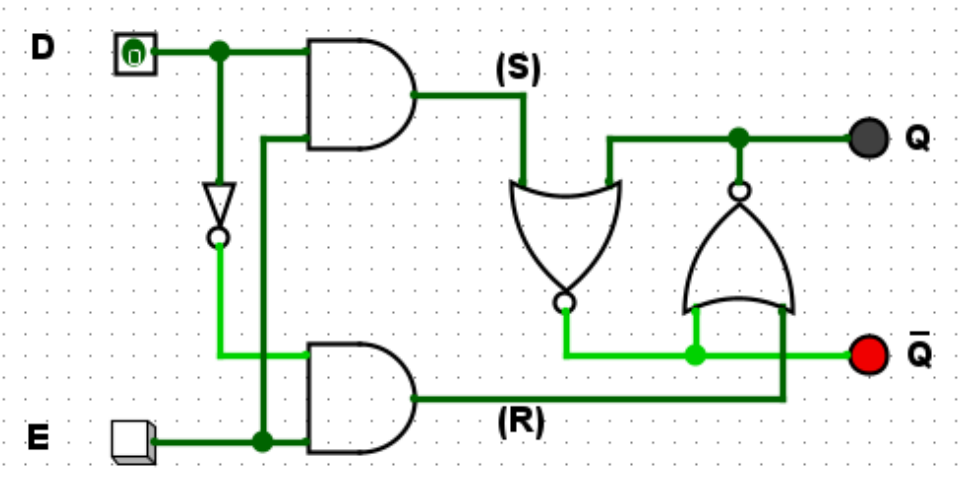
Die grundlegendste Form einer solchen Speicherzelle ist ein sogenannter SR-Flipflop, dieser hat zwei Eingänge, genannt S und R. S steht hierbei für *set* (zu Deutsch: setzen) und R für *reset* (zu Deutsch: zurücksetzen). Außerdem hat er zwei Ausgänge, genannt Q und $\overline{Q}$, die den gespeicherten Inhalt angeben. Q gibt den Inhalt wie gegeben an und $\overline{Q}$ gibt diesen in invertierter, also negierter Form an.
Eine einfache Möglichkeit dies umzusetzen, liegt darin, zwei NOR-Gates jeweils so zu verbinden, dass die Ausgabe des ersten Gates, A, in die Eingabe des zweiten Gates, B, geleitet wird und andersherum. Wenn nun das erste Gate, A, 1 bzw. Wahr ausgibt, hindert es das zweite Gate, B, damit daran, auch diesen Zustand anzunehmen, da es seine Ausgabe, die in diesem Fall 1 ist, in das zweite NOR-Gate, B, gibt und dieses somit unabhängig vom anderen Eingang nicht mehr Wahr ausgeben kann. Dies funktioniert auch genauso umgekehrt. Um nun die Ausgaben der Gates zu verändern, benutzt man die beiden Eingänge, die noch nicht benutzt wurden, denn die Ausgabe des NOR-Gates, das gerade noch 1 ausgibt, wechselt zu 0, wenn einer der Eingänge 1 ist. Man schaltet es um, indem man in den zweiten Eingang des gerade 1 ausgebenden NOR-Gates 1 gibt. Dies ändert nichts, wenn er schon aus ist. Also führt diese Mechanik dazu, dass der Flipflop nur den gespeicherten Wert ändert, wenn dieser nicht schon in diesen geändert wurde. Damit erfüllt dieser Schaltkreis genau die Funktion, die von einem SR-Flipflop erwartet wird[1]:

[1] Thormählen, "Sequentielle Schaltungen".



4.1.2 Pegelgesteuerter D-Flipflop

Um den Vorgang des Speicherns für den Computer kompatibel zu machen, baut man diesen Schalter zu einem D-Flipflop um. Ein D-Flipflop hat auch zwei Eingänge, man nennt sie D und E bei pegelgesteuerten D-Flipflops und D und C bei einem flankengesteuerten Flipflop. D steht hier in beiden Fällen für *data* (zu Deutsch: Daten - in diesem Fall ein einzelner Wert), der dann gespeichert wird. Da dies allerdings nur passieren soll, wenn es auch gewollt ist, gibt es den zweiten Eingang, im pegelgesteuerten Flipflop ist das E für *enable* (zu Deutsch: aktivieren), es wird nur der Wert, der über D eingegeben wird, gespeichert, wenn E aktiviert ist.
Das kann man einfach bauen, indem man einen SR-Flipflop nimmt, in den S-Eingang eine 1 eingibt, wenn D und E auch 1 sind, also schließt man ein AND-Gate vor, und tut bei dem R-Eingang dasselbe, nur, da man es hier für den gegenteiligen Fall baut, D negiert.[2]

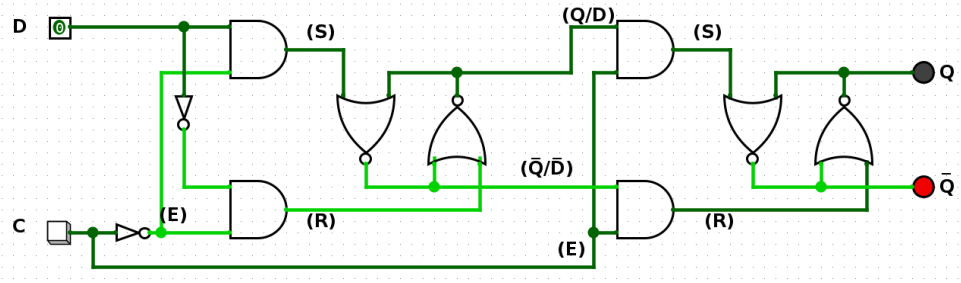


4.1.3 Flankengesteuerter D-Flipflop

Da es aber auch sein kann, dass man nun mit der jetzigen Computer-Operation z.B. den umgekehrten Wert oder einen anderen Wert, der sich mit der Ausgabe ändert, einspeichern

[2]Thormählen, "Sequentielle Schaltungen".

will, kann es zu Problemen kommen. Würde man nun z.B. einfach den $\overline{Q}$-Ausgang mit D verbinden, würde es so lange oszillieren wie man E aktiviert, und man würde nicht einfach den umgekehrten Wert einspeichern. Um das zu erreichen, soll der Flipflop nur einmal wechseln, und das macht man beim Wechsel des Signals von aus bzw. 0 auf an bzw. 1, diesen Wechsel des Signalstatus nennt man Flanke, dieser Wechsel von aus zu an nennt man positive Flanke. Daher heißt hier der zweite Eingang auch C für *clock* (sinngemäß: Taktgeber), da man hier meist einen Taktgeber anschließt.
Um dies technisch umzusetzen, gibt es mehrere Wege. Einer ist folgender. Man benutzt zwei pegelgesteuerte Flipflops. Der Erste speichert, solange der C-Eingang aus ist, den aktuellen Wert des D-Eingangs, hierfür kann man C einfach verneinen. Und der zweite übernimmt diesen Wert, wenn der C-Eingang aus ist. Da der erste Flipflop das Signal aber nicht mehr speichert, kann es auch nicht mehr weiter verändert werden, wenn sich die Eingabe an D ändert.[3]



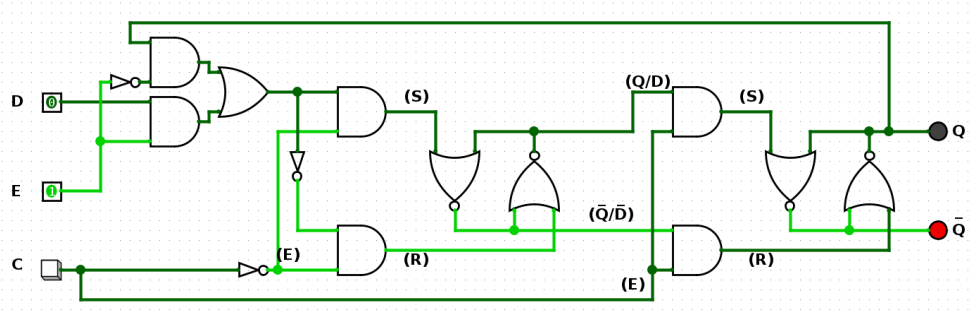
Man kann hier, wie man sieht, sich die Umkehrung am Eingang des D-Flipflops sparen, da man auch den sowieso schon verneinten zweiten Ausgang des ersten Flipflops anschließen kann.

4.1.4 Flankengesteuerter D-Flipflop mit Aktivierung

Häufig will man aber trotz des Takt-Signales noch eine zusätzliche Freigabe-Bedingung einbauen. Der D-Flipflop soll nur dann speichern, wenn der Takt von 0 zu 1 wechselt **und** der E-Eingang an ist. Intuitiv kommt man schnell auf die Idee, einfach C und E mit einem AND zusammenzuführen und in den vorherigen C-Eingang zu leiten. Diese Konstruktion bringt allerdings das Problem auf, dass während der Takt noch im an-Zustand ist, die Eingabe von E schon zum Speichern führt. Da das nicht passieren sollte, sondern nur bei der positiven Flanke des Taktes gespeichert werden soll, muss man einen anderen Ansatz wählen. Hierfür verändert man die D-Eingabe. Diese wechselt man abhängig von der E-Eingabe zwischen dem neu eingegebenen D und der momentanen Q-Ausgabe. Wenn E an ist, wird D in den Eingang geleitet und der Flipflop funktioniert so wie vorher auch, ist E allerdings aus, wird Q in den Eingang geleitet und bei positiven Flanken wird der aktuelle Wert, der schon im Speicher ist gespeichert, es ändert sich also nichts.
Eine Möglichkeit, diese Auswahlschaltung umzusetzen, sind zwei AND, die mit jeweils D und Q verbunden sind. In den zweiten Eingang führt man die E-Eingabe für das AND, mit der Q-Eingabe negiert man sie vorher. Nun kann nur das AND-Gate *an* sein, welches ein eingeschalteten zweiten Eingang hat, wenn also E *an* ist, das Gate mit D und sonst das mit Q. Dieses ausgewählte Gate, gibt nun, wenn der erste Eingang an ist auch *an* aus

[3] Thormählen, "Sequentielle Schaltungen".

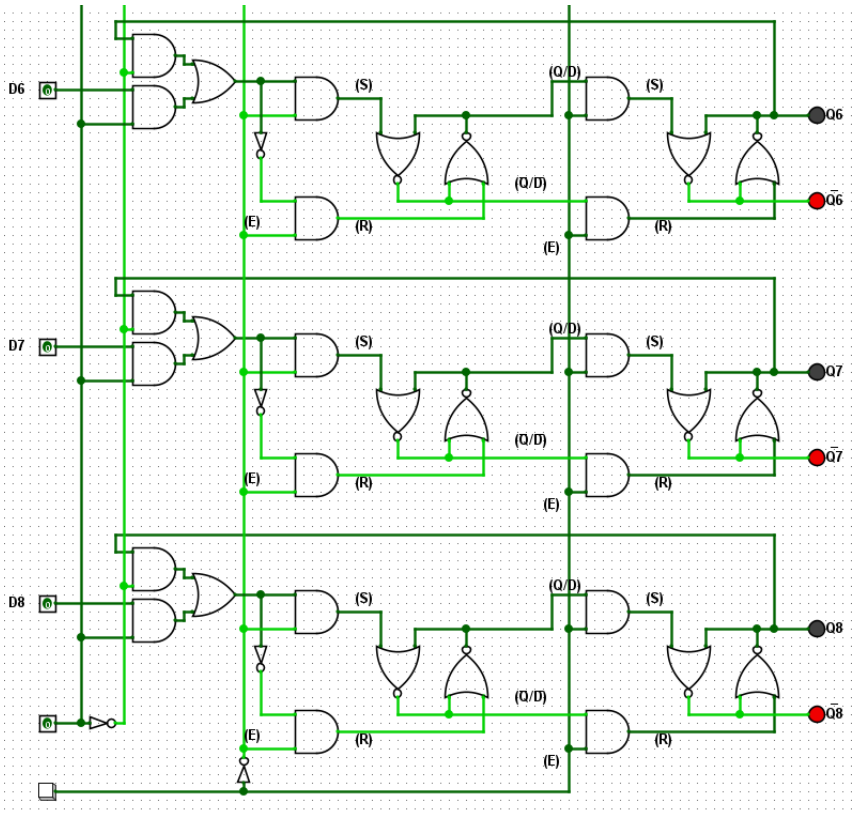
und sonst *aus*, es kopiert also diese Eingabe. Diese beiden Ausgaben verbindet man nun mit einem ODER-Gate.[4]



4.2 Verknüpfung mehrerer Speicherzellen

4.2.1 Register

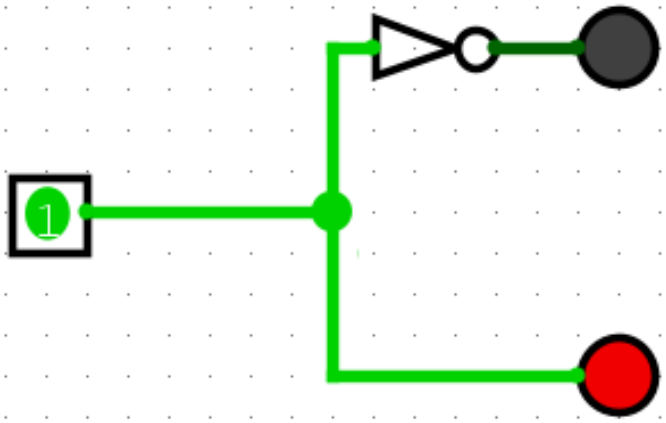
Da ein einzelnes Bit allein noch relativ wenig nützt, ordnet man meistens gleich mehrere Bits - Daten entsprechend nur einem Wahrheitswert: an und aus - in einem sogenannten Register an. Ein solcher Datensatz ist ein Speicherwort, in meinem Fall habe ich mich für acht Bits in einem Speicherwort entschieden. Diese acht D-Flipflops, die man zur Speicherung eines solchen Wortes braucht, aktiviert man alle über den gleichen Takt und dieselbe Eingabeaktivierung. Ein solcher Aufbau kann dann wie folgt aussehen - der obere Teil ist herausgeschnitten, da es sonst unübersichtlich groß wäre, es ist aber schlicht der gleiche Aufbau mehrmals wiederholt:



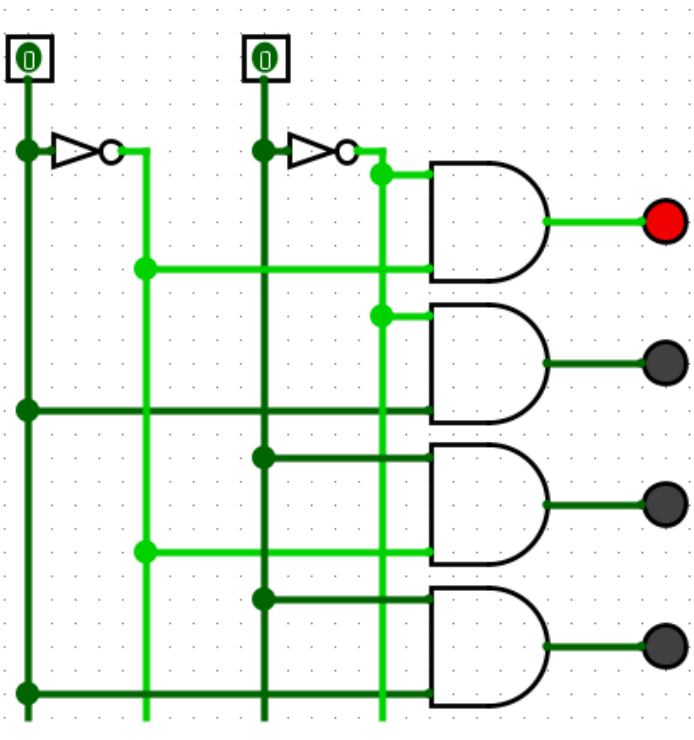
[4]Thormählen, "Sequentielle Schaltungen".

4.2.2 Addressierbarkeit mehrerer Register

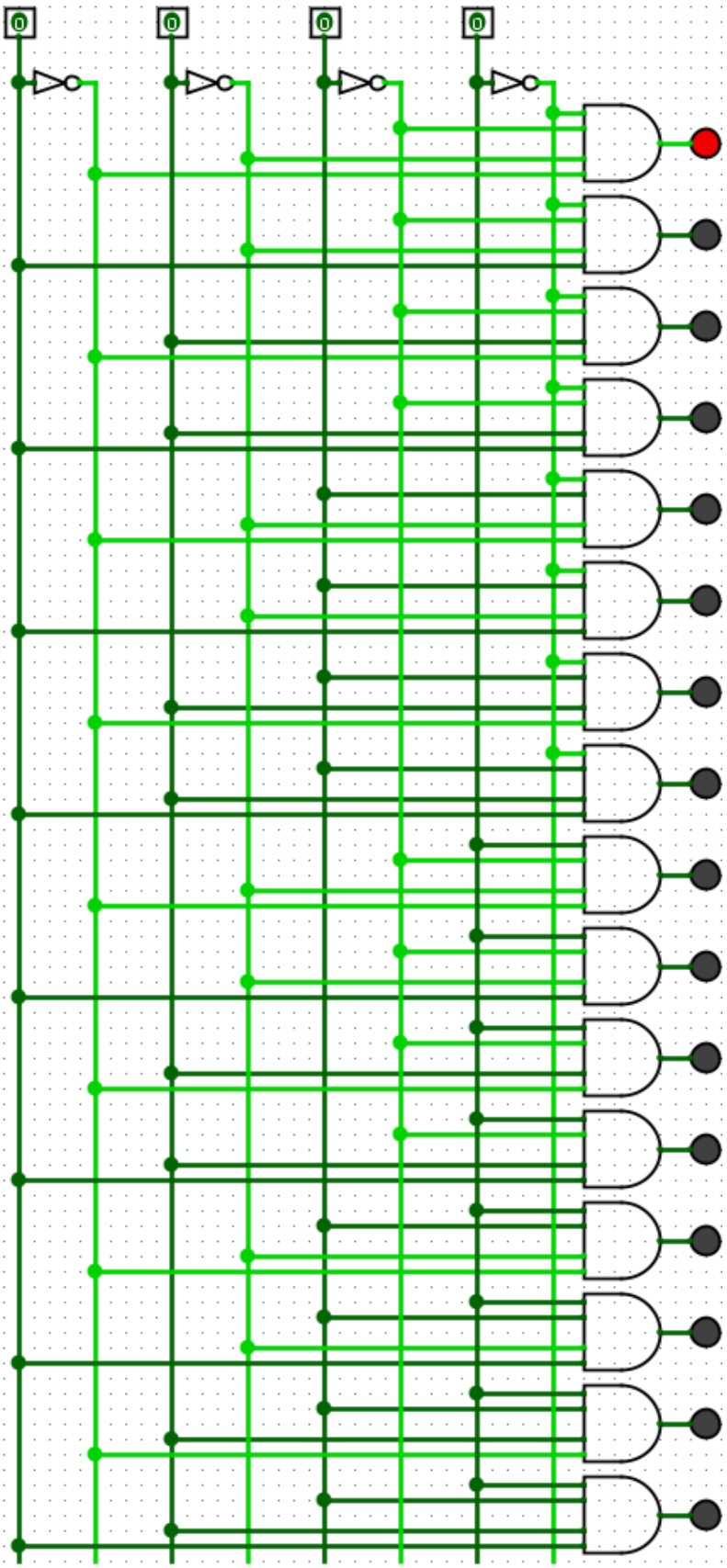
Auch ein einzelnes Register ist noch nicht ausreichend, um viel damit zu tun. Also benutzt man mehrere Speichereinheiten. Damit der Computer allerdings auch mit dieser Anzahl Register zurechtkommt, wird das Speicherwerk so entworfen, dass man von den vielen Registern gezielt auf ein bestimmtes zugreifen kann. Dafür weist man jedem Register eine Adresse zu, über welche diese dann sowohl beschrieben als auch gelesen werden kann. Bei zwei Registern ist die Adressierung noch relativ einfach, man hat die Adressen 0 und 1, das heißt, man aktiviert die eine Speichereinheit, wenn das Signal an ist und die andere, wenn das Signal aus ist, verneint also einen der E-Eingänge, welche hier dargestellt sind durch die roten Lampen:



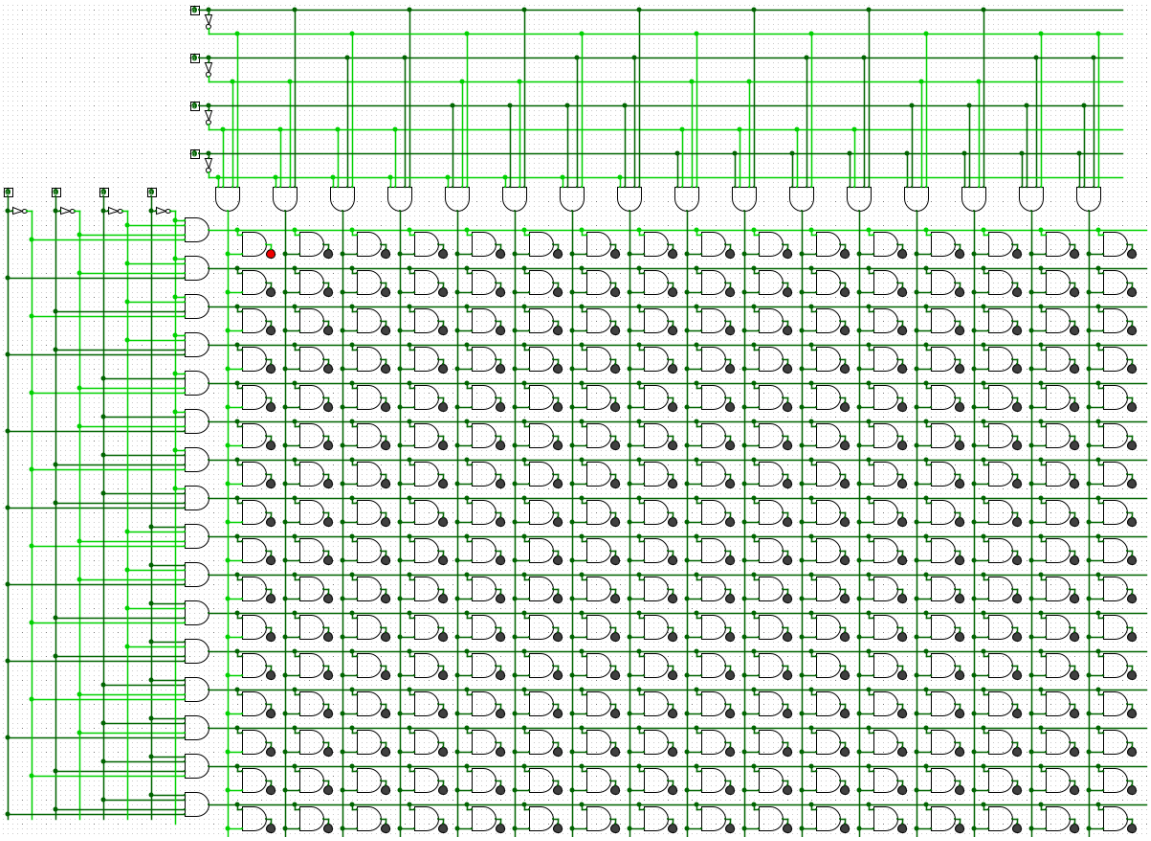
Wenn man nun zwischen vier unterscheiden will, kann man das auf ähnliche Weise umsetzen. Man kehrt auch hier die Eingaben um, allerdings muss man mit UND-Gattern noch jeweils die Kombinationen überprüfen. Ein solcher Dekodierer - so werden diese Arten von Schaltungen genannt - ist hier abgebildet:



Das kann man nun immer weiterführen, hier auf insgesamt 16 Register:



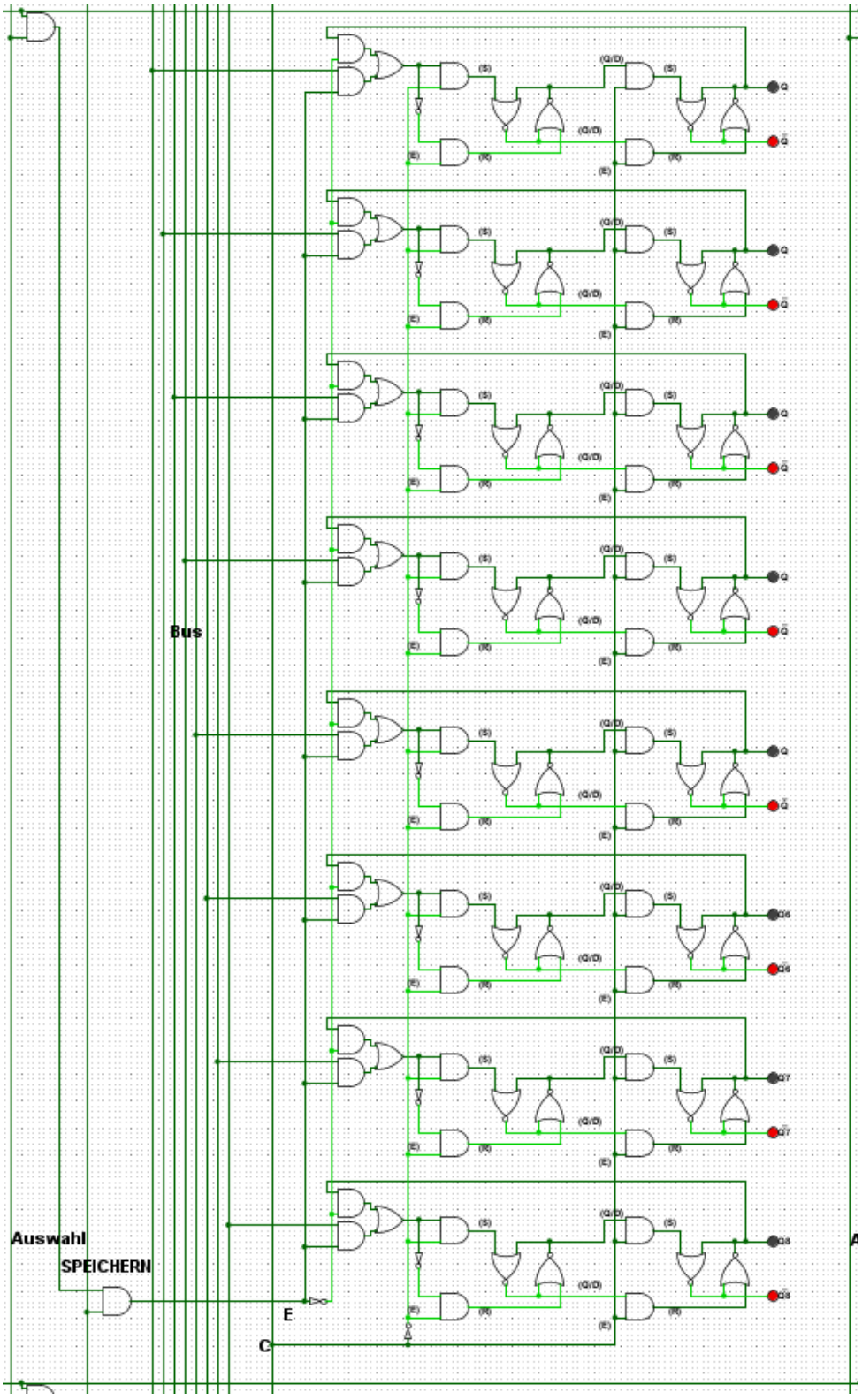
Um Material zu sparen, führt man diese nun im letzten Schritt in einem Gitter zusammen, in dem ein Dekodierer die Reihe und ein anderer die Spalte des ausgewählten Registers angibt. In den Zellen werden die beiden Signale schließlich durch ein UND-Gatter ausgelesen. Ein solcher Aufbau sieht dann in etwa so aus (auch hier repräsentieren die roten LED-Punkte die Register):



Diese Art der Vereinfachung hätte man schon beim Schritt vom 2bit- auf den 4bit-Dekodierer machen können, darauf habe ich allerdings aus Darstellungsgründen verzichtet.

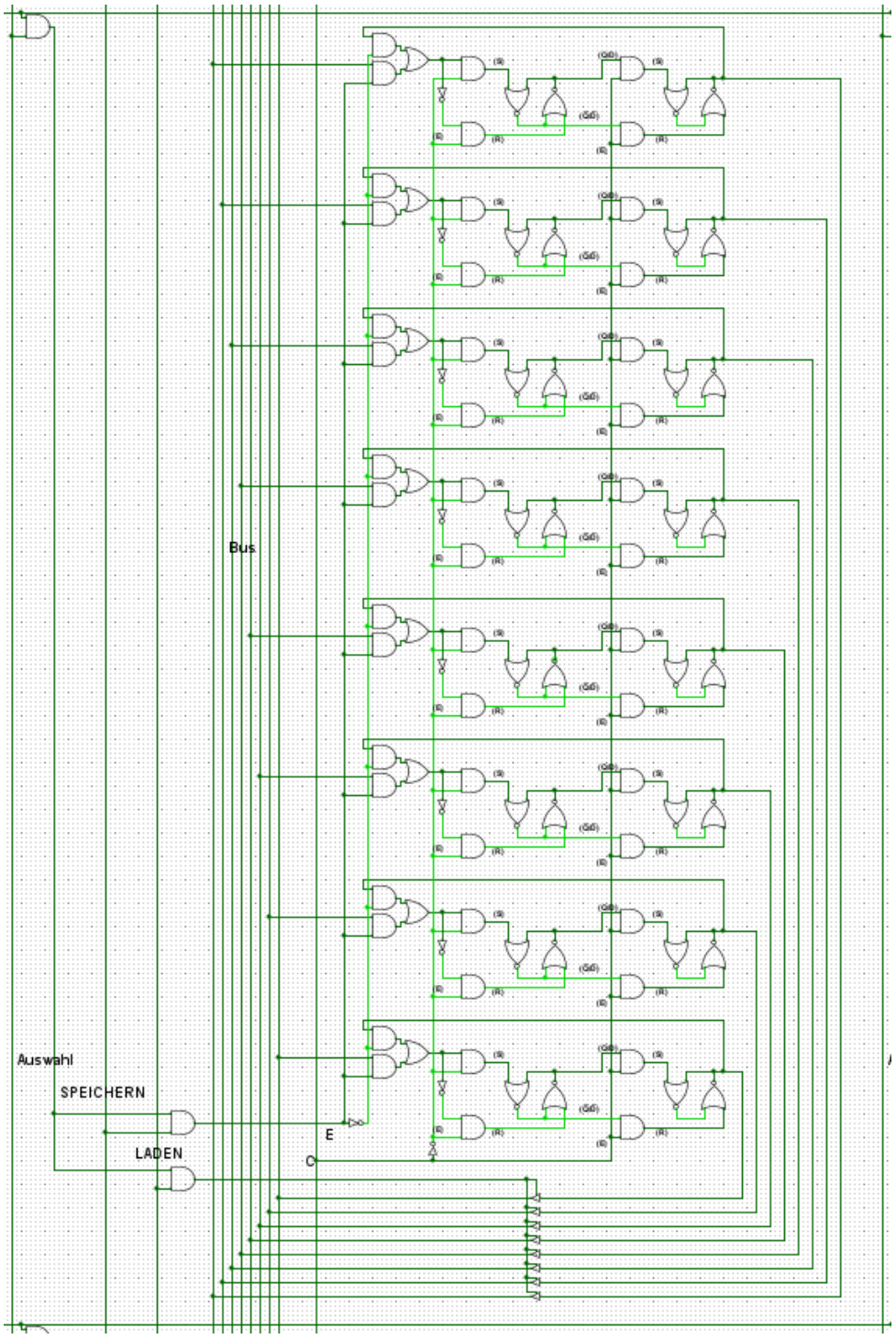
4.3 Verkabelung

Wie schon beschrieben, stecken hinter all den roten Lampen in Wahrheit Register. Diese sind jedoch nicht nur mit dem Signal aus dem Dekodierer verbunden, sondern vielmehr sind alle Register an den Bus angeschlossen, damit sie von dort Daten laden können. Damit sie dies allerdings nicht dauerhaft tun, sind sie mit auch mit dem an das Speicherwerk übergebene SPEICHERN-Signal verbunden (am E-Eingang). Damit aber nicht alle gleichzeitig speichern, wird dieses vorher noch durch ein AND-Gatter mit dem Auswahlsignal des Dekodierers kombiniert. Außerdem sind alle Taktgeber-Signale C zusammen geschaltet, damit alle gleichzeitig arbeiten. Der Inhalt einer Zelle im Dekodiergitter könnte also wie folgt aussehen:

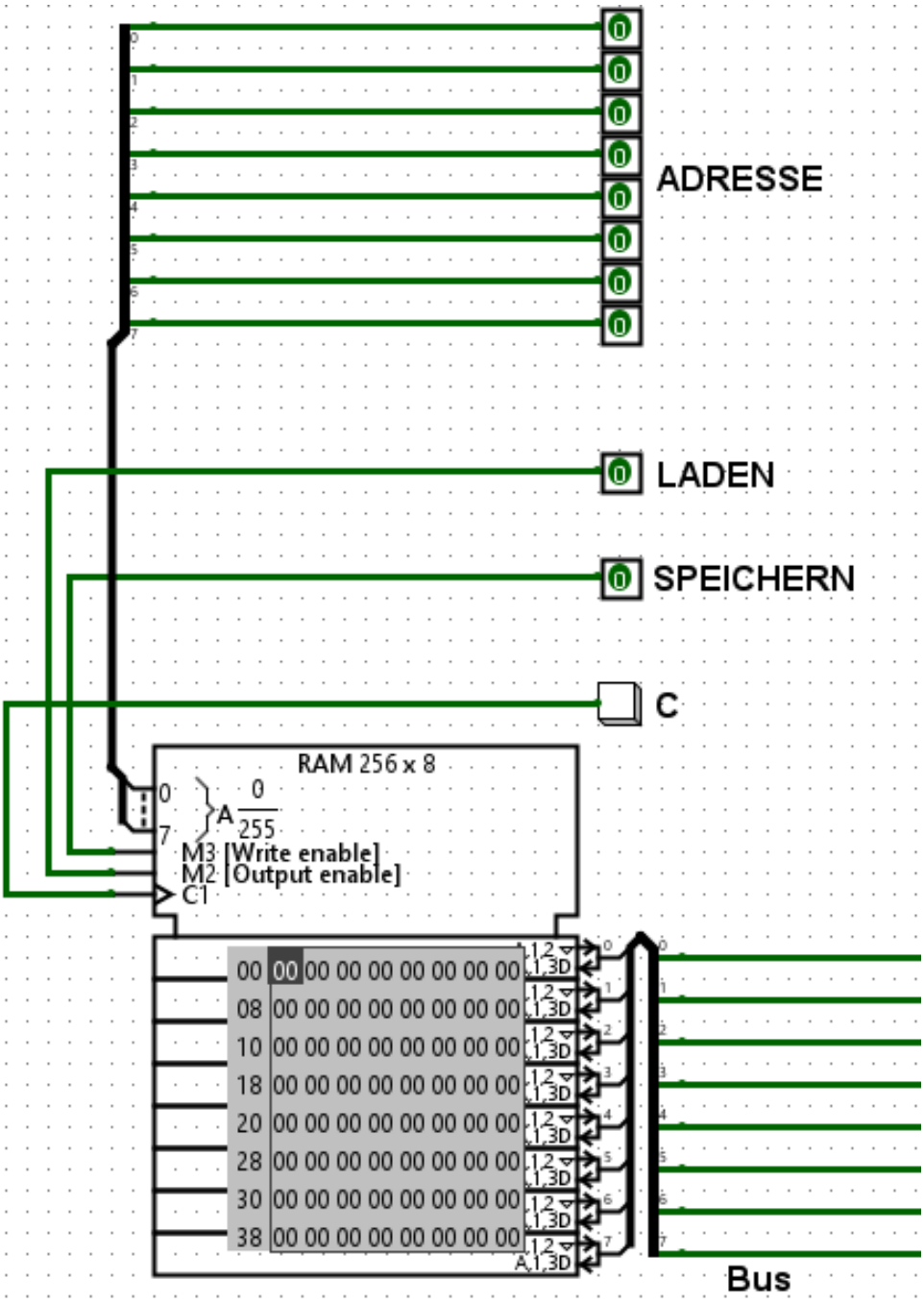


Allerdings ist der Sinn eines Speichers nicht nur, Daten zu speichern, sondern auch, sie wieder auszugeben. Daher sollen die Register auch auf dem Bus ihre Daten ausgeben können. Wie schon im Unterkapitel zu Bussen und der Tristate-Logik erläutert, geschieht eine solche Ausgabe auf den Bus immer über Tristate-Puffer. Die Aktivierungsbedingung für diese, also die Bedingung dafür, dass aus dem Speicher geladen wird, wird analog zum Speichern implementiert, nur mit dem Unterschied, dass dafür nicht das SPEICHERN-

Signal genutzt wird, sondern das LADEN-Signal:



Von diesen Registern sind nun im gesamten Speicherwerk 256 Stück eingebaut. Diesen ganzen Aufbau kann ich leider nicht zeigen, da das Programm, welches ich für die Schaltkreiszeichnungen benutze, bei 16 Stück in einem Gitter schon abgestürzt ist. Daher benutze ich im Folgenden den Speicherblock des Programmes, der wie folgt verkabelt wird:



Das ist schon das ganze Speicherwerk. Die Signale „ADRESSE“, „LADEN“, und „SPEICHERN“werden von der Steuereinheit bedient, darauf gehe ich im Kapitel zur Steuereinheit weiter ein. Das *C*-Signal ist das Taktsignal. Angeschlossen wird es an einen Taktgeber, der an alle Komponenten angeschlossen ist, damit diese alle parallel operieren können.

Kapitel 5

Das Rechenwerk

5.1 Dem Computer Zahlen erklären

Wir stellen Zahlen üblicherweise im Dezimalsystem dar. Da ein Computer aber nur an und aus kennt, bietet sich für diesen Zweck das Binärsystem an. Während man im Dezimalsystem zehn Ziffern hat (0, 1, ..., 8, 9), benutzt man im Binärsystem nur die zwei Ziffern 1 und 0. 1 wird als an und 0 wird als aus dargestellt, so wird dieses System auch dem Computer greifbar. Hierbei benutzt man als Ziffernwerte nicht Einer, Zehner, Hunderter, Tausender, ..., also die Zehnerpotenzen (10^n), sondern die Zweierpotenzen (2^n), so hat man Einer, Zweier, Vierer, Achter, Sechzehner, ... So wird z.B. aus 67 1000011 und andersherum 101010 zu 42.

5.2 Addieren

Beim Addieren kann man sich am schriftlichen Addieren orientieren. Die jeweiligen Ziffern an denselben Stellen der Zahlen addiert man einzeln und addiert gegebenenfalls einen Übertrag zu den nächsten Ziffern:

$$\begin{array}{cccc} & 5 & 4 & 8 \\ + & 5 & 1 & 5 \\ & {\scriptstyle 1} & & {\scriptstyle 1} \\ \hline 1 & 0 & 6 & 3 \end{array}$$

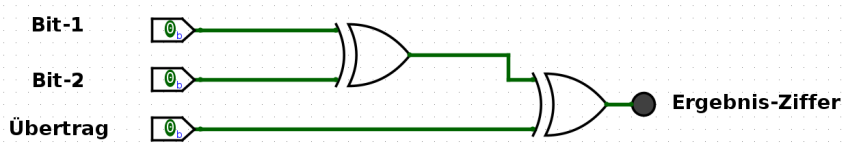
Analog dazu verfährt man im Binärsystem:

$$\begin{array}{cccccccc} & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ + & & 1 & 1 & 0 & 0 & 1 & 1 \\ & {\scriptstyle 1} & {\scriptstyle 1} & & {\scriptstyle 1} & {\scriptstyle 1} & {\scriptstyle 1} & \\ \hline 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{array}$$

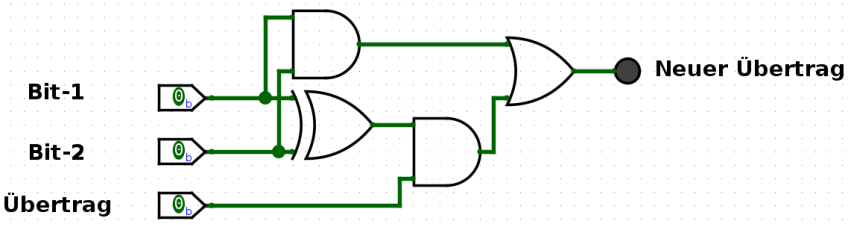
Also kann man die Operation des Addierens in mehrere Schritte des Addierens der jeweiligen Ziffern und eventuellen Überträge aufteilen. Ein solcher Schaltkreis, der dies tut, müsste also zu der folgenden Wahrheitstabelle passen:

Ziffer 1	Ziffer 2	vorheriger Übertrag	Ziffer Summe	neuer Übertrag
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

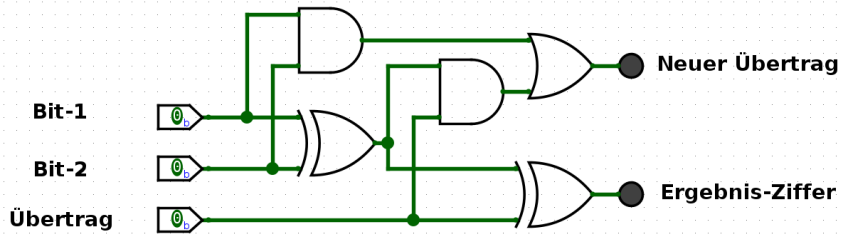
Man erkennt leicht, dass die neue Ziffer der Summe durch ein einfaches drei-Eingaben-XOR berechnet werden kann:



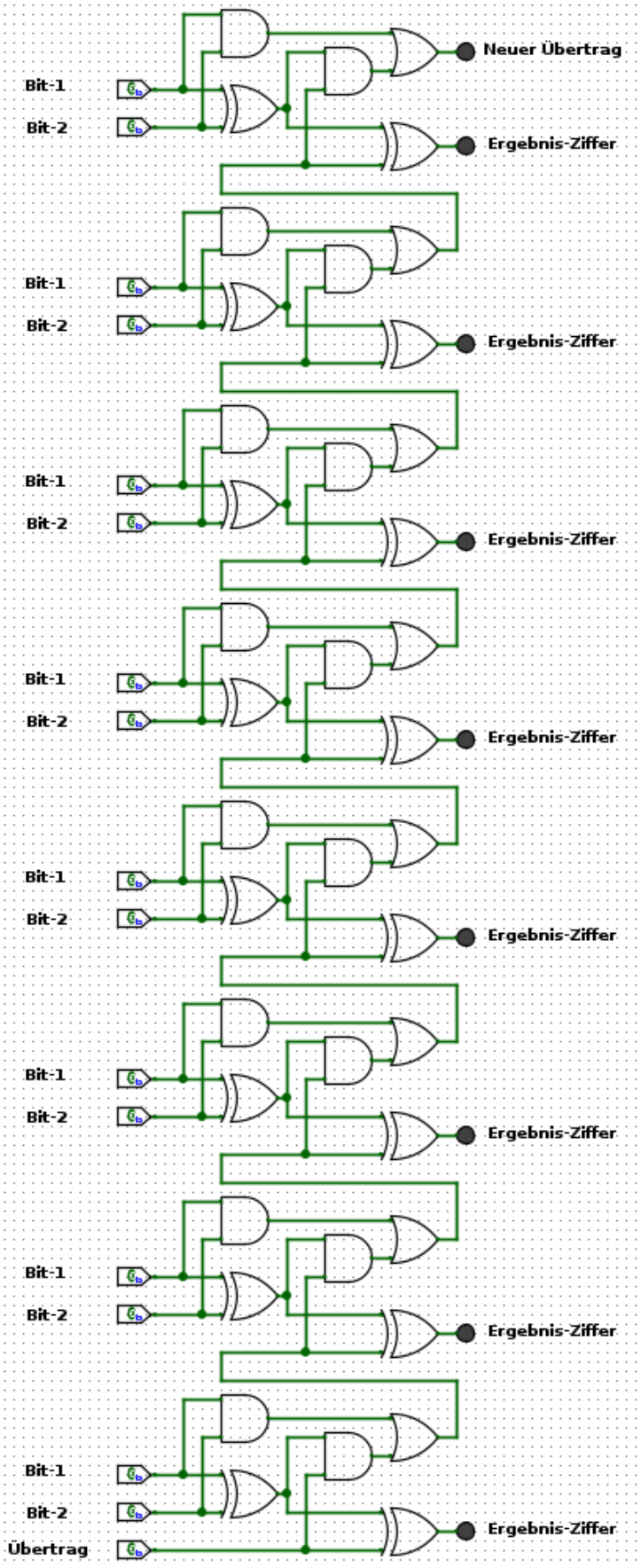
Nun muss man zur Berechnung des neuen Übertrags überprüfen, ob mindestens zwei der drei Eingänge an sind. Das kann man am einfachsten prüfen, indem man überprüft, ob die erste und zweite Ziffer 1 ist oder nur eine der beiden Ziffern 1 ist, aber auch der Übertrag 1 ist. Ein Aufbau, der so agiert, kann so aussehen:



Bei der Zusammenführung kann man sich schließlich sogar ein XOR-Gatter sparen, da beide Aufbauten ein XOR-Gatter beinhalten, das die beiden Ziffern als Eingang nimmt. Man kann die Ausgabe eines Gatters in beide Gatter geben, die diese weiterverarbeiten:



Diese Schaltungen kann man nun aneinanderreihen, um gleich zwei ganze Speicherwörter zu addieren. Hierbei schließt man den Übertrag-Ausgang jedes Addierers immer an den Übertrag-Eingang des jeweils nächsten an. Ausgenommen hiervon ist natürlich der letzte Addierer, welcher keinen Nachfolger hat.


Neuer Übertrag
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Ergebnis-Ziffer
Bit-1
Bit-2
Übertrag
Ergebnis-Ziffer

5.3 Subtrahieren

Zum Subtrahieren benutze ich das System des Zweierkomplements[1], eine Möglichkeit der Darstellung negativer Zahlen. Dieses hat den Vorteil, dass man negative Zahlen genau so addieren kann wie auch positive Zahlen, mit einem Addierer wie oben beschrieben. Bei diesem System negiert man den vordersten Stellenwert. Da dieser Computer mit Speicherwörtern, die acht Bits beinhalten - acht binäre Ziffern, wird in diesem Fall der Ziffernwert 2^7 also 128 durch -128 ersetzt. Also ist nun nach diesem neuen Stellenwertsystem 10010110 nicht mehr

$$1 \cdot 128 + 0 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 0 \cdot 1$$

$$= 150$$

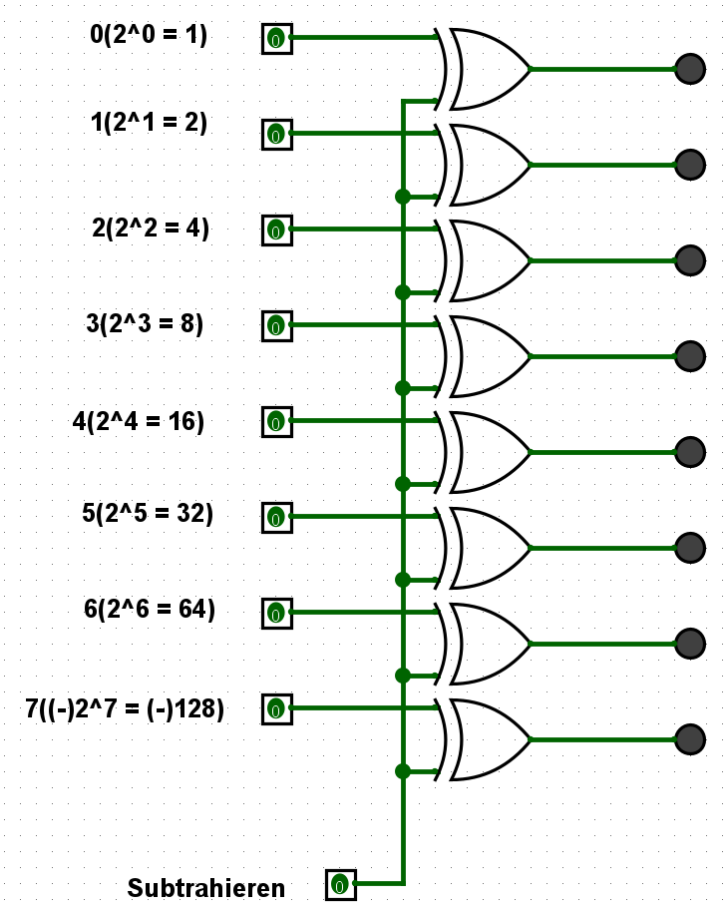
sondern

$$1 \cdot (-128) + 0 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 0 \cdot 1$$

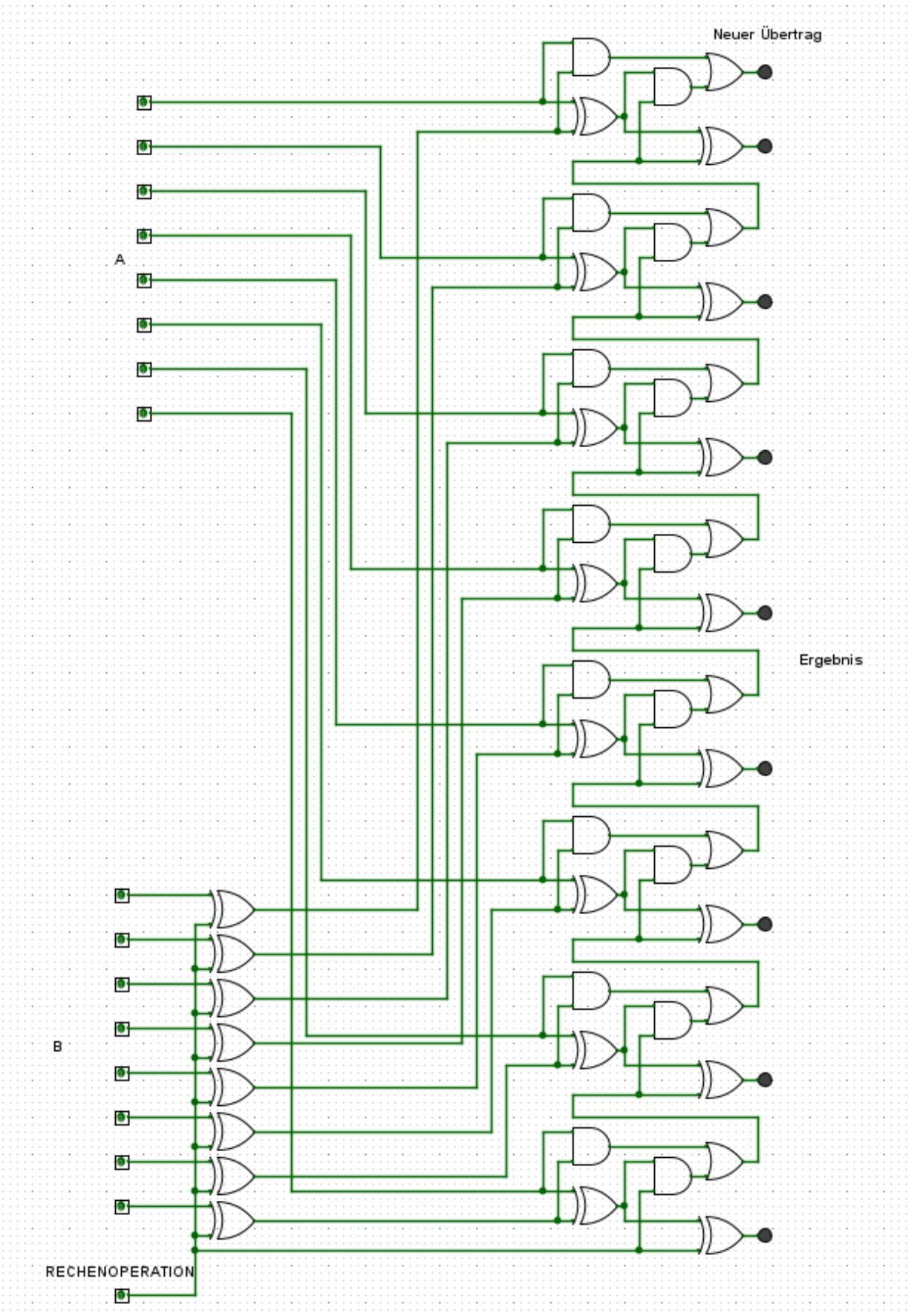
$$= -106$$

Dadurch kann man nun alle Zahlen in der Spanne von -128 (10000000) bis 127 (01111111) darstellen. Diese kann man nun wie schon erwähnt durch den schon gebauten Addierer mit anderen Zahlen addieren, zum Subtrahieren muss man also nur vor den Addierer noch eine Einheit schalten, die die zu addierende Zahl negiert, Dazu kann man einen einfachen Trick anwenden.
Man invertiert alle Bits und addiert schließlich 1.[2] Warum das funktioniert, möchte ich in diesem Kontext nicht explizit erklären. Es ist in der angegebenen Quelle allerdings genauer erläutert. Um dies umzusetzen, betrachtet man im ersten Schritt das Invertieren der Bits, das entspricht einfach der Verneinung der Bits. Da man den Subtraktionsmodus allerdings an- und ausschalten können soll, kann man das besser mit XOR-Gattern machen, diese fungieren nämlich für die erste Eingabe gesehen als Verneinung genau dann, wenn die zweite an ist:

[1] Finley, "Two's Complement".
[2] Finley, "Two's Complement".



Um das Addieren mit 1 durchzuführen, gibt man das Subtrahieren-Signal auch als Übertrag in den Addierer, der Übertrag ist nämlich nichts anderes als eins zu addieren:

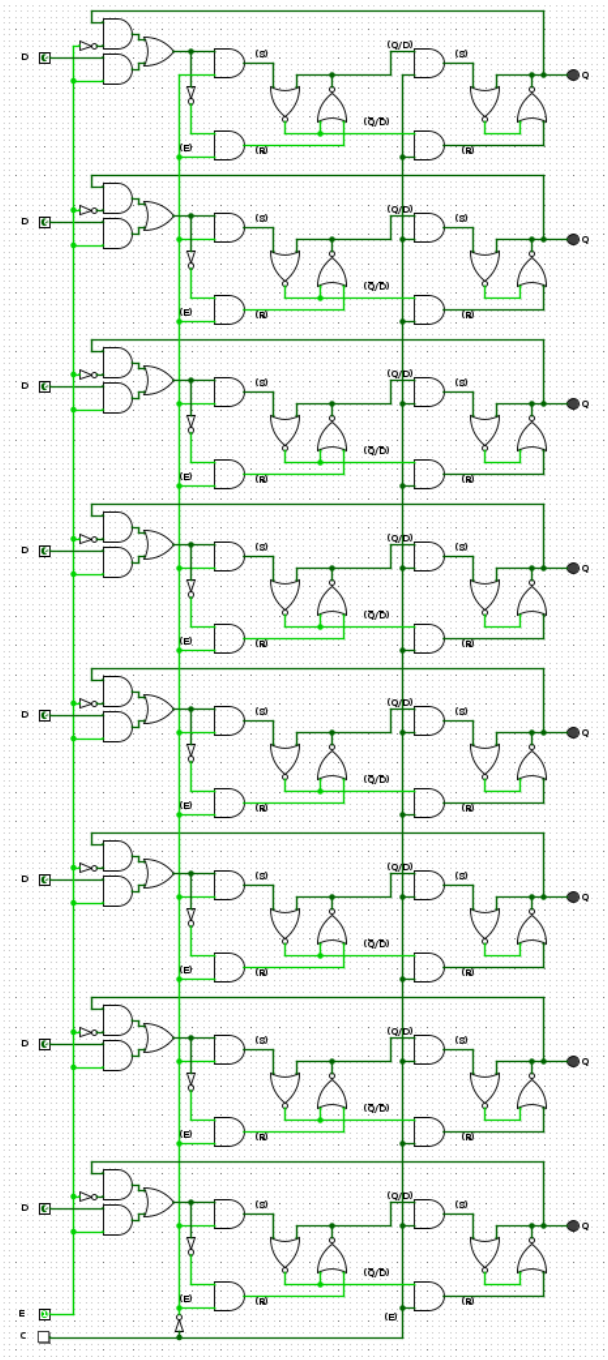


Das ist bereits die komplette Einheit, die wirklich rechnet, diese einfache Version kann

fürs Erste nur addieren und subtrahieren. Welche der Operationen ausgeführt wird, bestimmt das RECHENOPERATIONS-Signal. Ist dieses ausgeschaltet, entspricht das Ergebnis $A + B$ ist es hingegen eingeschaltet $A - B$. Ein Rechenwerk beinhaltet allerdings außerdem noch einen sogenannten Akkumulator, dieser ist ein Speicherregister, das immer die aktuelle Zahl speichert.

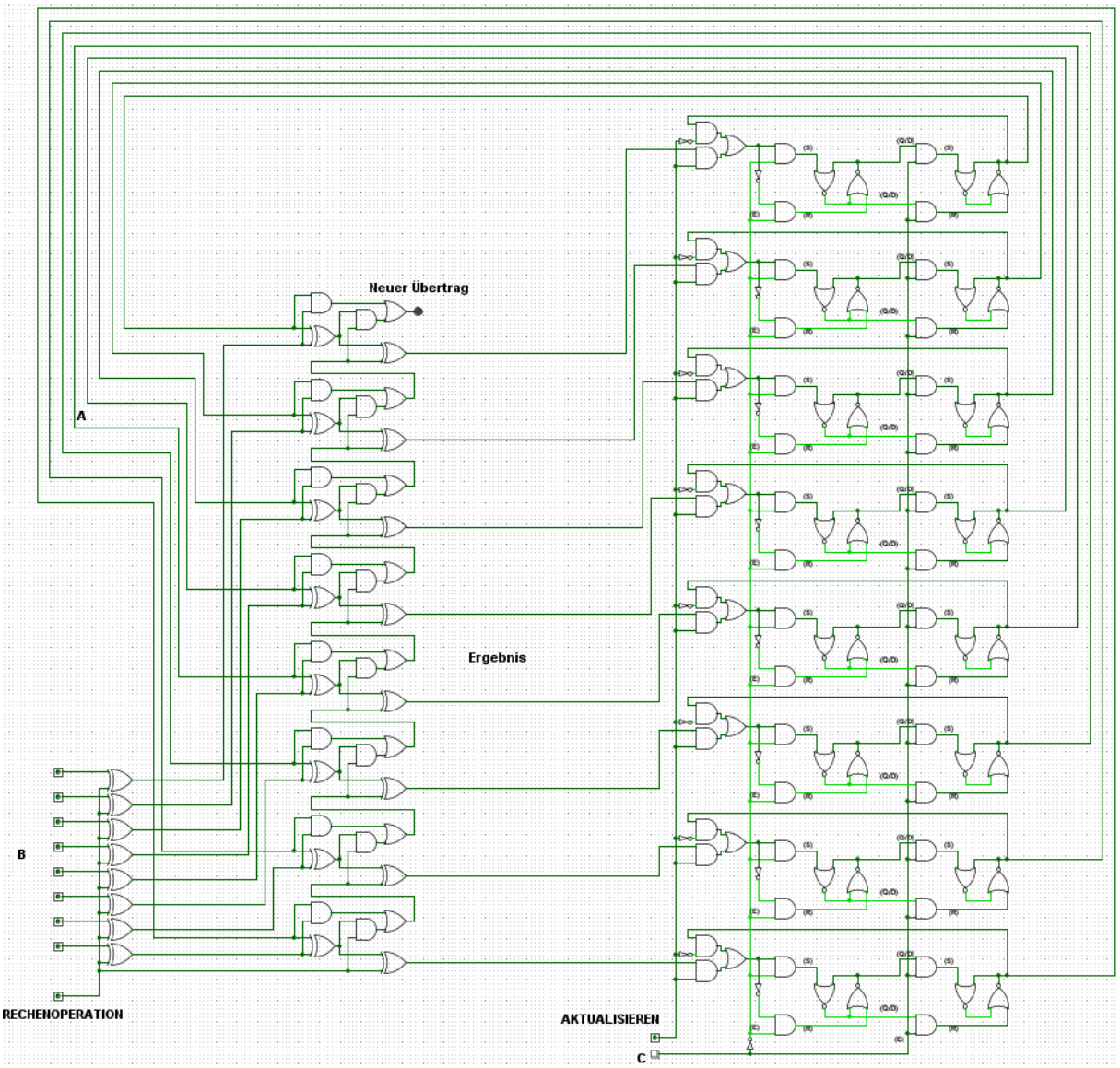
5.4 Akkumulator

Der Akkumulator ist ein einfaches Speicherregister, wie es schon oben im Kapitel zum Speicherwerk erläutert wird:

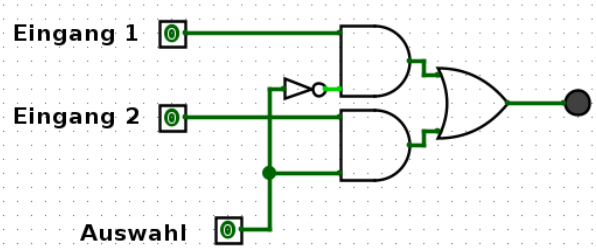


Hier ist ein solches Register jetzt im Ganzen und ohne verneinten Ausgang abgebildet.

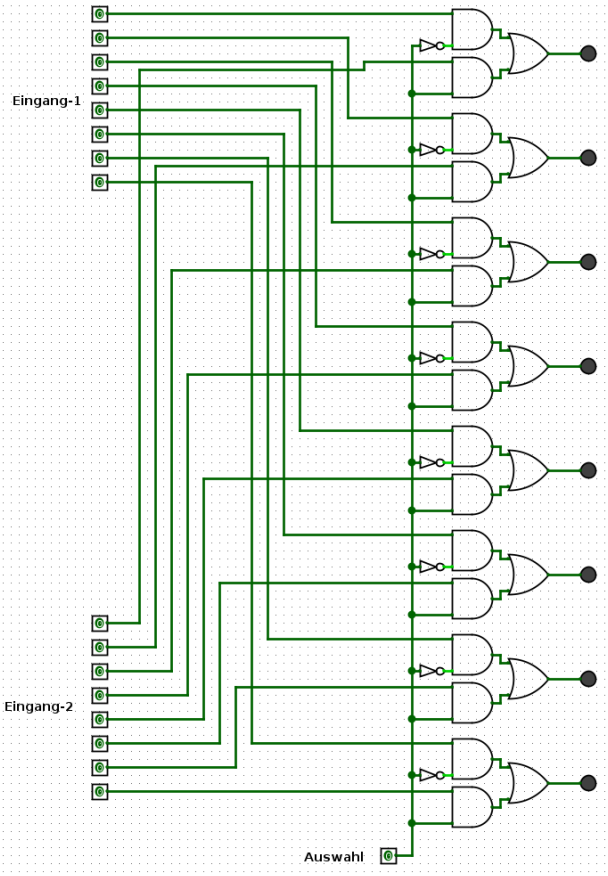
Der Akkumulator ist an die Recheneinheit angeschlossen, sodass seine Ausgabe immer in den Eingang A von ihr geht und die Ausgabe der Recheneinheit immer in seinen Eingang:



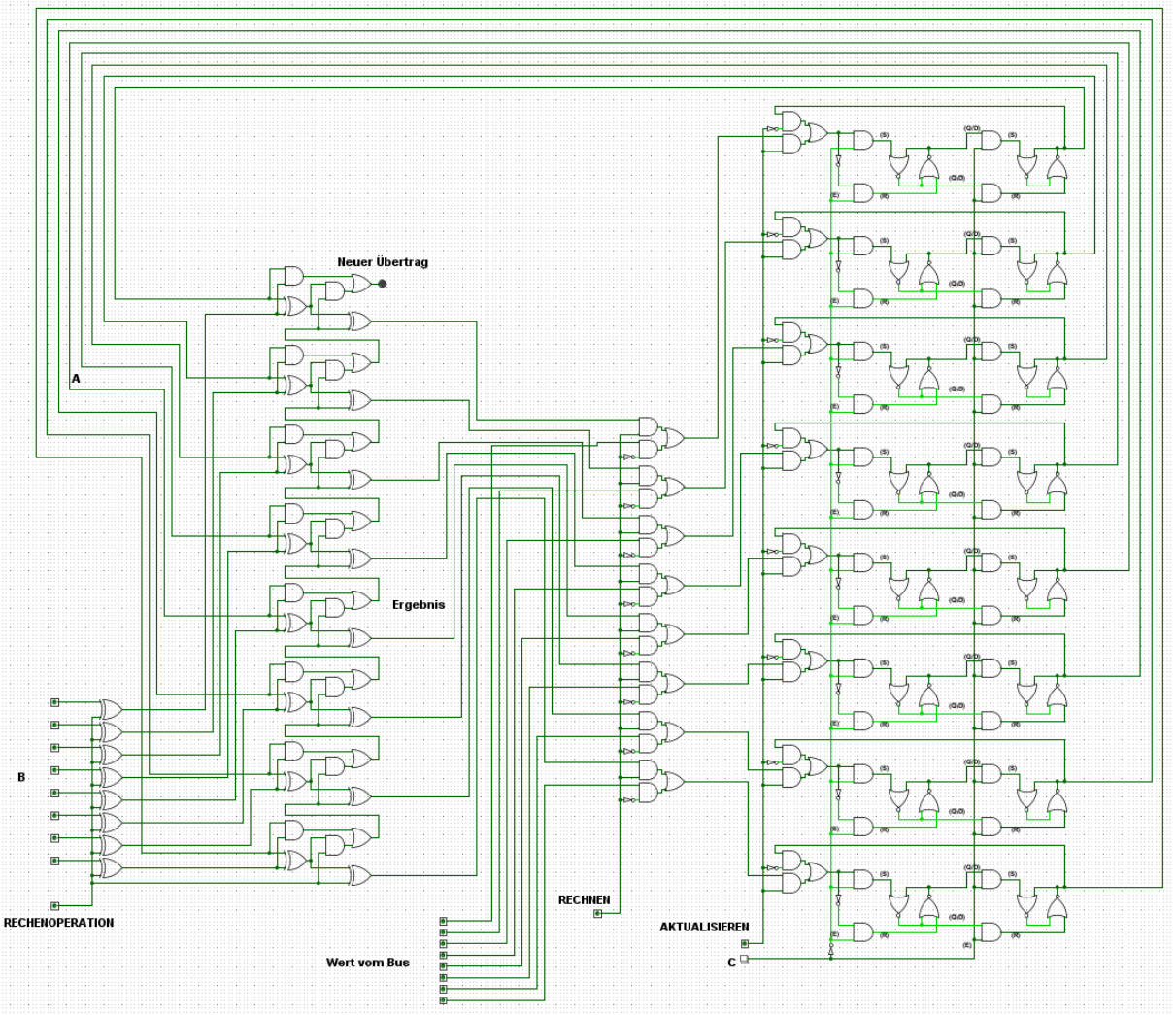
Allerdings muss der Akkumulator auch Daten von dem Bus laden können. Daher muss eine Schaltung gebaut werden, die entscheidet, ob die Ausgabe der Recheneinheit oder der Inhalt des Busses in den Akkumulator geladen wird.
AND-Gatter können dafür benutzt werden, da genau dann der Zustand des ersten Eingangs ausgegeben wird, wenn der zweite Eingang eingeschaltet ist, und sonst aus/0 ausgegeben wird. Dies nutzt man, indem man die beiden auszuwählenden Signale in die ersten Eingänge der AND-Gatter führt und das Auswahl-Signal in die zweiten Eingänge der AND-Gatter. Damit nicht beide immer gleichzeitig ausgewählt werden, negiert man eines der Signale in den zweiten Eingang. Am Ende führt man die Ausgaben der AND-Gatter mit einem OR-Gatter zusammen:



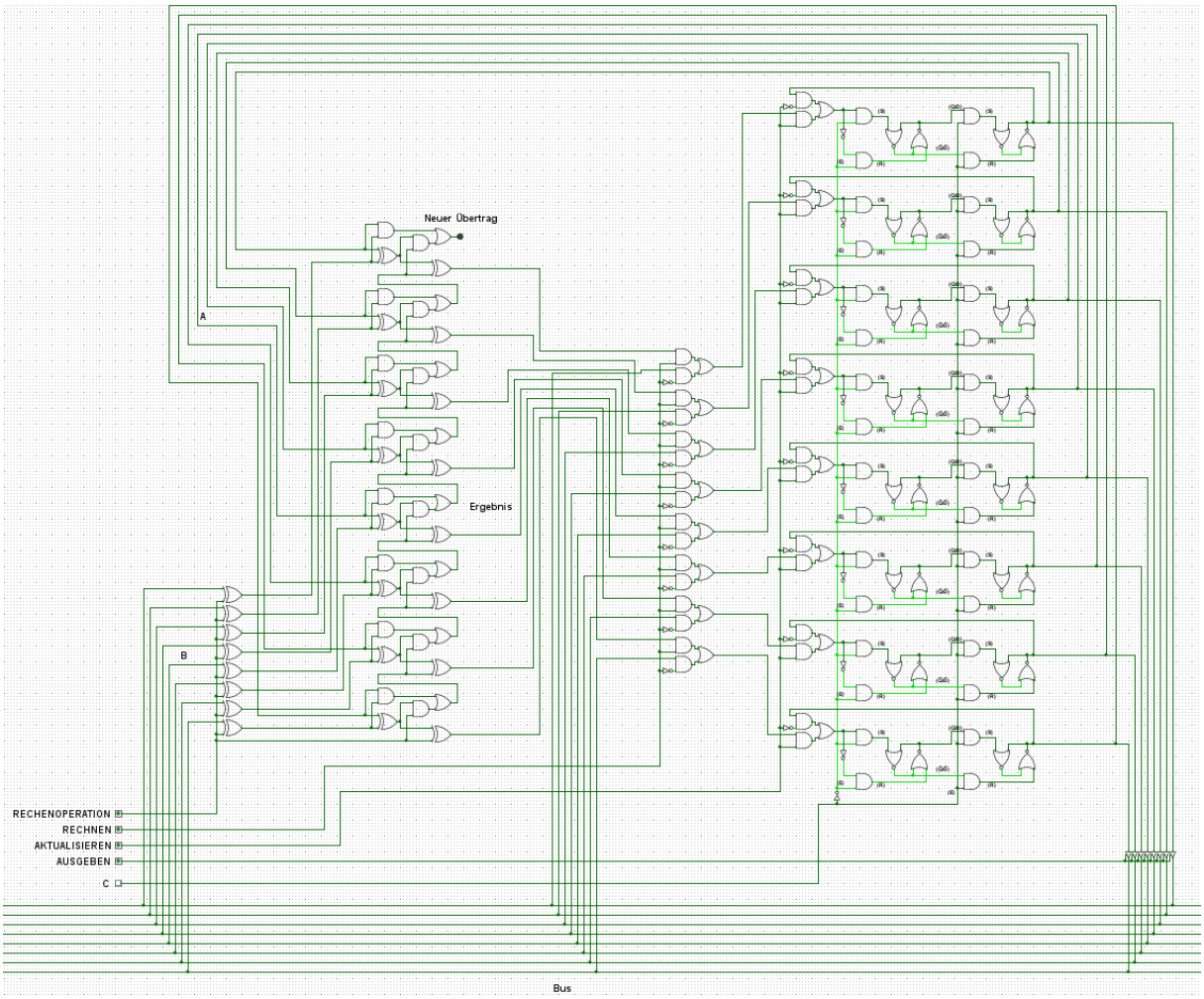
Das kann man auf acht Bits erweitern, indem man acht dieser Schaltungen zusammenschaltet:



Mit dieser Auswahlschaltung kann man nun ermöglichen, dass der Akkumulator sowohl aus der Recheneinheit als auch von dem Bus laden kann, bei dieser Auswahlschaltung sind die NOT-Gatter an anderer Stelle, damit der Wert vom Bus dann importiert wird, wenn das Auswahlsignal „RECHNEN“aus istund nicht andersherum:



Nun muss nur noch die Verkabelung zum Rest des Computers geregelt werden. Dafür wird sowohl B als auch der „Wert vom Bus“-Eingang mit dem Bus verbunden. Außerdem sollte die Recheneinheit auch etwas ausgeben können, da sonst die errechneten Werte umsonst berechnet wurden. Deshalb kriegt das Rechenwerk die Möglichkeit, den Akkumulatorinhalt auf den Bus zu geben. Dies muss, wie beim Speicher schon getan, wieder mit Tristate-Puffern gemacht werden:



Das ist zunächst schon das ganze Rechenwerk. Die Signale „RECHENOPERATION“, „RECHNEN“, „AKTUALISIEREN“und „AUSGEBEN“werden von der Steuereinheit bedient, dazu im folgenden Kapitel mehr. Das C-Signal ist wie beim Speicherwerk auch an den Taktgeber angeschlossen, der an alle Komponenten angeschlossen ist, damit diese alle parallel operieren können. Der „Neuer Übertrag“-Ausgang ist noch nicht wichtig, er wird erst benötigt, wenn man mit Zahlen rechen will, die man auf mehrere Speicherwörter aufteilen muss. Darauf werde ich im Rahmen dieses Projektkurses jedoch nicht eingehen.

Kapitel 6

Das Steuerwerk

6.1 Funktionen des Steuerwerks

Das Steuerwerk ist das Kernstück des Rechners. Es koordiniert alle anderen Komponenten. Dafür braucht es vor allem Anschlüsse an die Steuereingaben aller anderen Komponenten. Außerdem muss es je nachdem welcher Befehl gerade eingegeben wird, diese verschieden senden.

6.2 Befehle

Diese verschiedenen Operationen führt es, je nach Befehl, mit verschiedenen Steuerungssignalen durch. So muss z.B. bei einem Befehl zum Laden des Inhaltes einer Speichereinheit die jeweilige Adresse der Speichereinheit übermittelt werden und deren Ausgabe aktiviert werden.
Bevor das nun umgesetzt wird, muss man sich zuerst noch über die Darstellung klar sein, in der man die Befehle benutzt. Ich werde hierfür die folgende Darstellung verwenden und einen Befehl immer als zwei Zahlen darstellen, da der Computer nur Zahlen wirklich interpretieren kann. Die kann er nämlich, wie schon im Kapitel zum Rechenwerk erwähnt, in Form der Binärdarstellung interpretieren. Die erste Zahl gibt an, welche Art von Befehl ausgewählt ist, dafür wird jedem Befehl ein Zahlcode zugeordnet, über den man ihn dann eindeutig identifizieren kann. Die zweite Zahl gibt eine weitere Information als Parameter an, z.B. bei dem Laden aus dem Speicher, welche Adresse im Speicher geladen werden soll.

Folgende Befehle will ich fürs Erste einbauen:

Befehl	Zahlcode	Parameter	Funktion
Lade	0	Adresse aus der geladen werden soll	Lädt den Wert in den Akkumulator, der in der Speicherzelle hinterlegt ist
Speicher	1	Adresse in die gespeichert werden soll	Speichert den Wert des Akkumulators in den Speicherwerk
Addiere	2	Adresse dessen Inhalt addiert werden soll	Addiert Wert aus dem Speicherwerk auf den Akkumulator
Subtrahiere	3	Adresse dessen Inhalt subtrahiert werden soll	Subtrahiert Wert aus dem Speicherwerk vom Akkumulator

Wie man merkt, ist ein Befehl zum Laden eines festen Wertes nicht vorhanden. Das liegt daran, dass ein solcher in der Implementation sehr aufwändig wäre, während er sich auch einfach dadurch realisieren lässt, dass man in einem Computerprogramm solche Werte neben den Befehlen in den Speicher schreibt.

6.3 Das Verbinden der Komponenten

Um nun die oben genannten Befehle zu implementieren, muss man sich vorher darum kümmern, die Komponenten richtig zu verkabeln. Hierbei geht es nun auch um die konkrete Planung jedes Signals, nicht, wie im Kapitel zum Aufbau der von-Neumann-Architektur, nur um einen groben Plan, wie die Komponenten kommunizieren.

6.3.1 Die Kabel des Speicherwerks

Um das Speicherwerk zu instruieren, muss das Steuerwerk diesem übermitteln können, unter welcher Adresse geladen bzw. gespeichert werden soll und vor allem, ob gespeichert werden soll und auch ob der Inhalt der übergebenen Adresse ausgegeben werden soll. Denn diese Ausgabe des Speicherwerks ist an den Datenbus angeschlossen. Auf diesen kann jede Komponente Daten geben und die darüber übertragenen Daten laden. Genauer erläutert wird das im Unterkapitel zu Bussen und der Tristate-Logik.
Das heißt, für das Speicherwerk brauchen wir insgesamt ein Adresssteuersignal, ein Steuersignal zum Speichern und ein Steuersignal zum Ausgeben.

6.3.2 Die Kabel des Rechenwerks

Das Rechenwerk braucht einerseits ein Steuersignal, ob der Akkumulator sich aktualisieren soll, ob dies direkt vom Bus kommen soll oder von einer durchgeführten Berechnung und dann auch welche Berechnung es sein soll - in dem Fall des hier entworfenen sehr primitiven Rechenwerks nur Addieren oder Subtrahieren. Zudem, damit die Berechnungen auch weiter verwertet werden können, muss das Rechenwerk auch wissen, ob es den Akkumulatorinhalt auf den Bus ausgeben soll.
Zusammengefasst braucht man also für das Rechenwerk je ein Signal für das Aktualisieren des Akkumulators, ob gerechnet werden soll, welche Rechenoperation ausgeführt werden soll und ob der Akkumulatorinhalt ausgegeben werden soll.

6.3.3 Alle Steuersignale

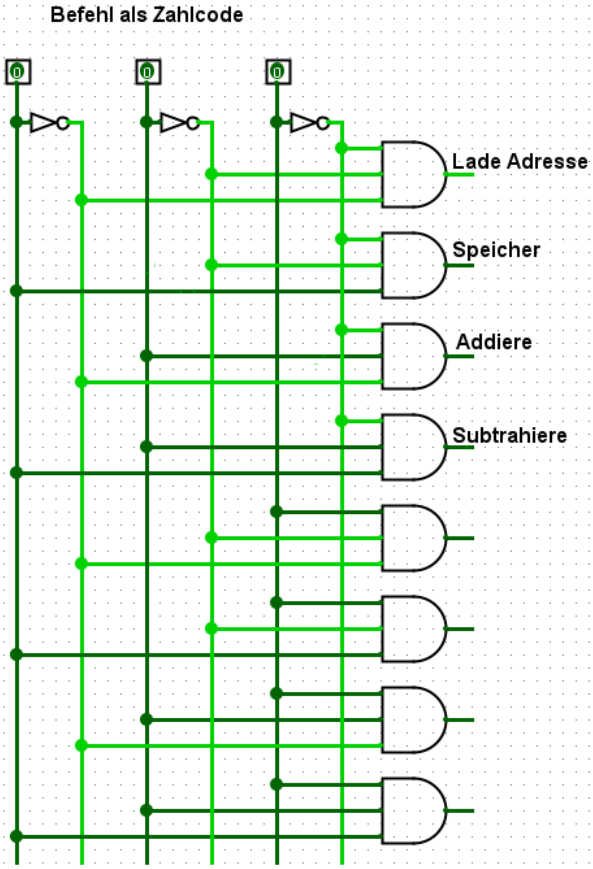
Das sind schon alle Steuersignale, die für das Steuerwerk bis hierhin nötig sind, um den Speicher und das Rechenwerk zu steuern:

1. ADRESSE - die Auswahl des Speicherregisters im Speicherwerk
2. SPEICHERN - Businhalt im Speicher speichern
3. LADEN - Inhalt des ausgewählten Speicherregisters auf den Bus geben
4. AKTUALISIEREN - Akkumulator aktualisieren

5. RECHNEN - das Rechenwerk berechnet den neuen Wert, statt ihn vom Bus zu laden
6. RECHENOPERATION - wechselt zwischen Addieren und Subtrahieren
7. AUSGEBEN - gibt den Akkumulatorinhalt auf den Bus aus

6.4 Dekodieren der Befehle

Die Befehlseingabe wird hier als Zwischenschritt händisch ausgeführt. Später wenn der Computer Programme ausführen soll, werden diese Befehle, aus denen die Programme bestehen, automatisch aus dem Speicher geladen. Als erster Schritt wird der Zahlcode dekodiert. Das heißt es wird eine Zahl n in einem Zahlenraum $0 \leq n < m$ in Binärdarstellung so umgewandelt, dass von m Ausgängen der $n$te Ausgang aktiviert ist, wobei man bei der Nummerierung, wie immer in der Informatik bei 0 anfängt. Einen solchen Aufbau habe ich schon im Kapitel zum Speicherwerk im Unterkapitel zur Adressierbarkeit mehrerer Register erläutert. Hier habe ich einen solchen Aufbau für 3 Eingänge also für $2^3 = 8$ Ausgänge gebaut - mehr Befehle werden es nicht. Mit diesem Aufbau hat man nun für jeden Befehl ein Signal, das angibt, ob dieser der gerade auszuführende Befehl ist. Abhängig von diesen Signalen kann man nun die passenden Steuersignale senden:



6.5 Implementation der Befehle

Jetzt kann man sich einfach für jeden einzelnen Befehl überlegen, welche Steuersignale eingeschaltet sein müssen. Da der zweite Teil des Befehls immer eine zu behandelnde

Adresse ist, kann dieser direkt an das Adresssignal des Speicherwerkes angeschlossen werden. Daher muss dieser nicht besonders beachtet werden.

6.5.1 Lade-Befehl

Der Lade Befehl muss zuerst den Speicher instruieren, den Inhalt des ausgewählten Speicherregisters auf den Bus zu geben. Also muss das LADEN-Signal eingeschaltet werden. Dieser Speicherinhalt, der nun auf dem Bus ist, muss anschließend im Akkumulator gespeichert werden. Dazu muss das Signal AKTUALISIEREN an sein. Die restlichen Signale bleiben ausgeschaltet, außer natürlich wie schon erwähnt das ADRESS-Signal.

6.5.2 Speicher-Befehl

Beim Speichern in das Speicherwerk muss der Vorgang genau andersherum ablaufen, der Akkumulatorinhalt muss auf den Bus ausgegeben werden. Dies wird durch das AUSGEBEN-Signal instruiert. Dann muss das Speicherwerk diese Daten auch speichern, dies wird ihm durch das Signal SPEICHERN angewiesen.

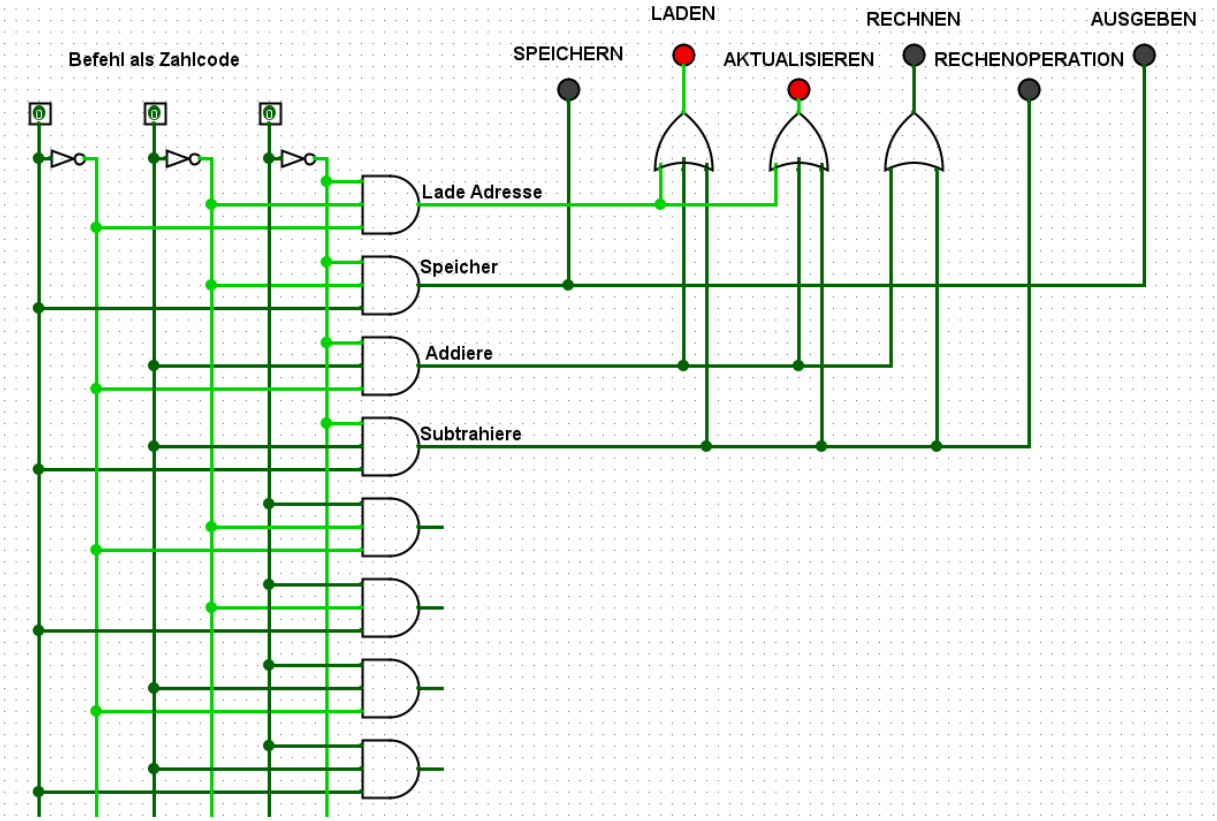
6.5.3 Addiere-Befehl

Beim Addieren ist es zunächst wichtig, dass das Speicherwerk die aufzuaddierende Zahl auf dem Bus zur Verfügung stellt. Diese Anweisung erhält es durch das LADEN-Signal. Das Rechenwerk muss nun folgendermaßen instruiert werden, es muss den Akkumulatorinhalt aktualisieren, also muss das Steuerwerk das AKTUALISIEREN-Signal einschalten. Außerdem muss das Signal RECHNEN an sein, damit gerechnet und nicht nur vom Bus geladen wird, und das Signal RECHENOPERATION muss aus sein, damit addiert und nicht subtrahiert wird. Die restlichen Signale sind auch aus.

6.5.4 Subtrahiere-Befehl

Der Subtrahiere-Befehl agiert analog zum Addiere-Befehl. Es gibt hier nur einen Unterschied beim Signal RECHENOPERATION, dieses ist nämlich bei diesem Befehl eingeschaltet, damit das Rechenwerk vom Addiermodus in den Subtrahiermodus wechselt.

Diese Steuerlogiken kann man nun dadurch realisieren, dass man die dekodierten Befehlssignale mit OR-Gattern auf die Steuersignale führt oder dort, wo sowieso nur ein Befehl an das Signal angeschlossen werden muss, diesen auch direkt anschließt:

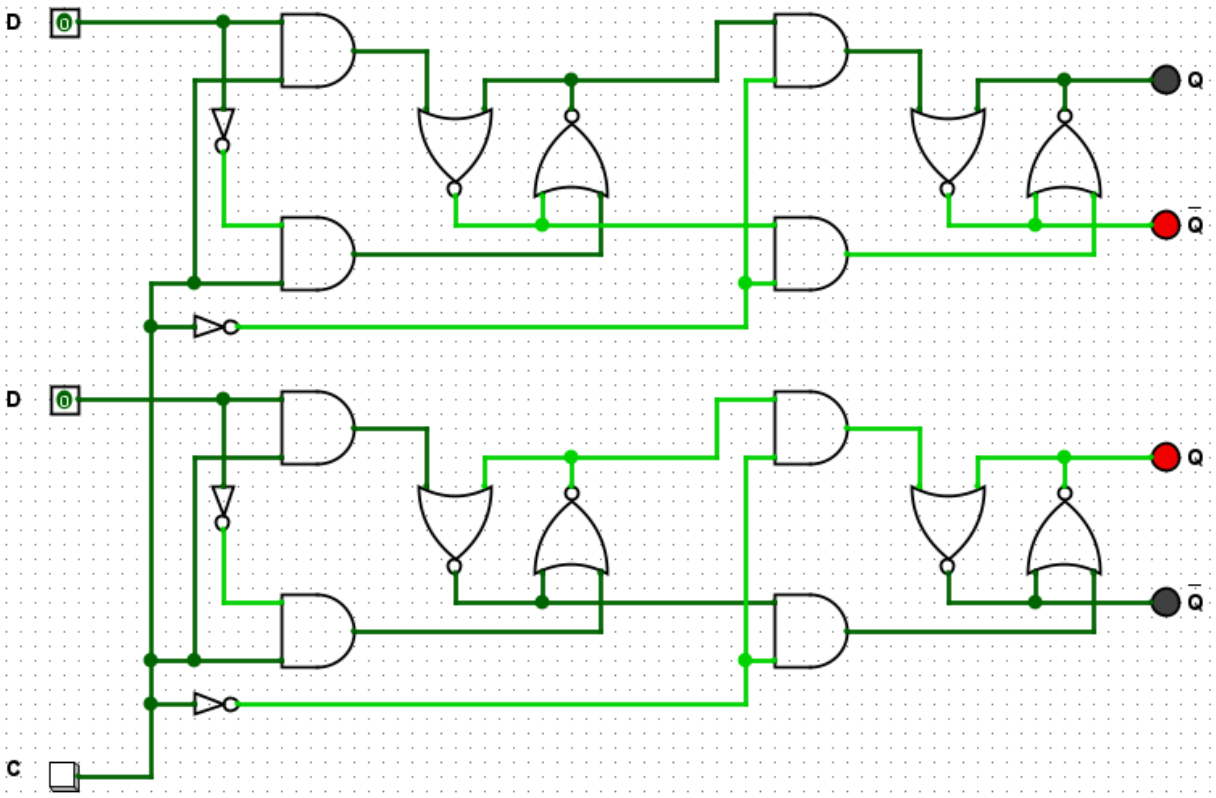


Wenn man dieses Steuerwerk nun in den Computer einbauen würde, könnte man diese Befehle schon ausführen, indem man den Befehl als Zahlencode und den Adressteil des Befehls eingibt und anschließend das Taktsignal betätigt.

6.6 Befehle aus dem Speicher laden

Eine solche Bedienung des Computers ist aber noch längst nicht das, was man von einem Computer erwarten würde. Ein solcher sollte nämlich auch von sich aus Befehle ausführen können. Diese muss er aus dem Speicherwerk laden - zumindest bei der von mir gewählten von-Neumann-Architektur. Bei der Harvard-Architektur z.B. gibt es dafür einen separaten Speicher[1].
Das hat den Nachteil, dass dieser Vorgang mehrere Taktzyklen beansprucht. In meinem Computer ist das folgender Vorgang: Im ersten Takt wird der erste Teil des Befehls, der Zahlcode, geladen. Im zweiten Takt schließlich der zweite Teil, der Adressteil, geladen. Im dritten Takt wird darauf folgend der Befehl ausgeführt. Dann beginnt dieser Ablauf mit dem nächsten Befehl von vorne.
Um dies zu realisieren, benötigt es vor allem einen Speicher, der sich merkt, in welchem der drei Schritte sich der Computer befindet. Ein solcher Speicher kann man durch zwei D-Flipflops realisieren, die Flanken gesteuert sind, allerdings kein Aktivierungssignal benötigen. Solche Flipflops sind bereits oben in dem Kapitel Flankengesteuerter D-Flipflop erläutert:

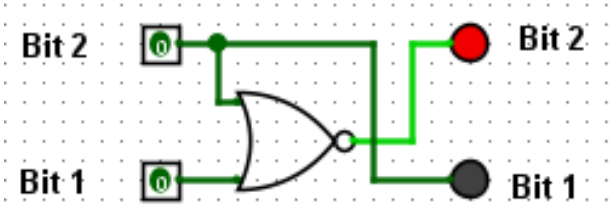
[1] Franz, "Rechnerarchitektur".



Die NOT-Gatter befinden sich hier am zweiten Pegelgesteuerten D-Flipflop statt am ersten, damit sie auf der fallenden Flanke sich aktualisieren; auf der steigenden Flanke werden nämlich alle Berechnungen etc. ausgeführt. Das funktioniert, da dies äquivalent dazu ist, den C Eingang zu negieren. Denn die steigende Flanke von $\overline{C}$ ist die fallende Flanke von C.
Um nun automatisch von Schritt 1 bzw. 0 (00) zu Schritt 2 bzw. 1 (01) zu schalten und schließlich von Schritt 2 bzw. 1(01) zu Schritt 3 bzw. 2(10) zu schalten und wieder zurück, benötigt man einen Schaltkreis, der folgender Logiktabelle folgt:

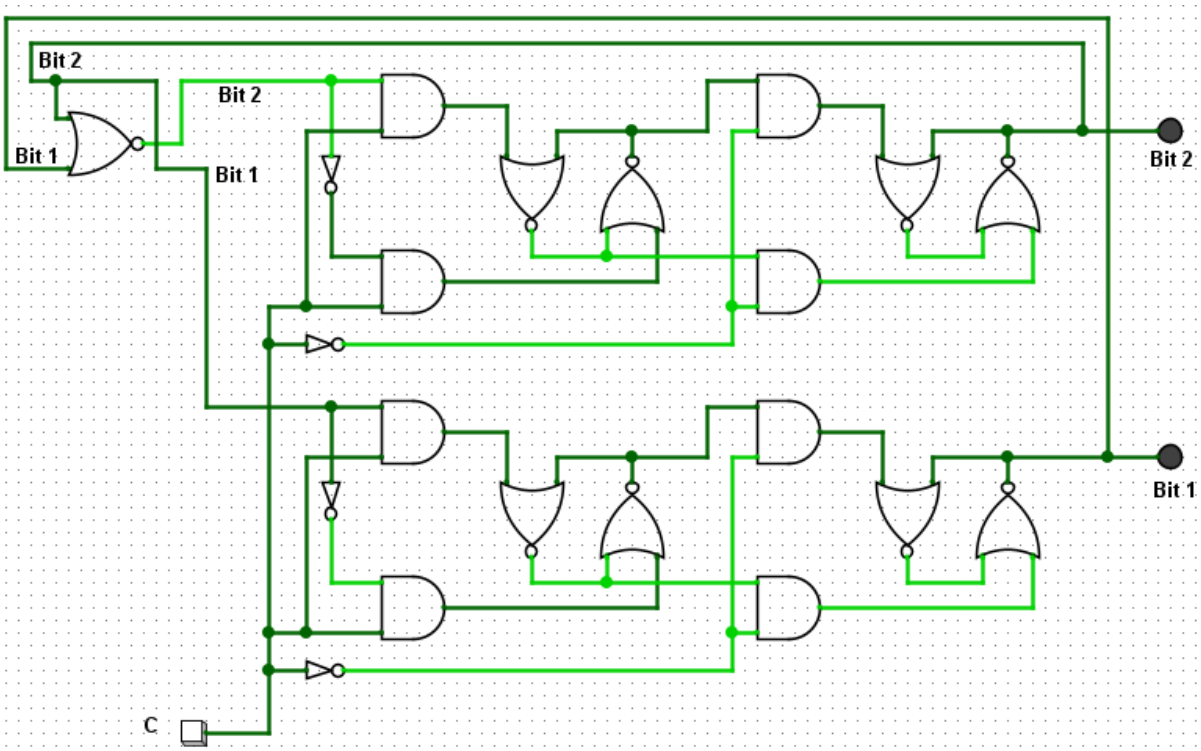
Bit 1 alt	Bit 2 alt	Bit 1 neu	Bit 2 neu
0	0	0	1
0	1	1	0
1	0	0	0

Der neue Bit 2 ergibt sich, wie man sieht, durch ein einfaches NOR-Gatter der beiden alten Bits. Der erste Bit ist sogar noch einfacher und ergibt sich direkt aus dem vorherigen zweiten Bit. Als Schaltkreis lässt sich das wie folgt aufbauen:



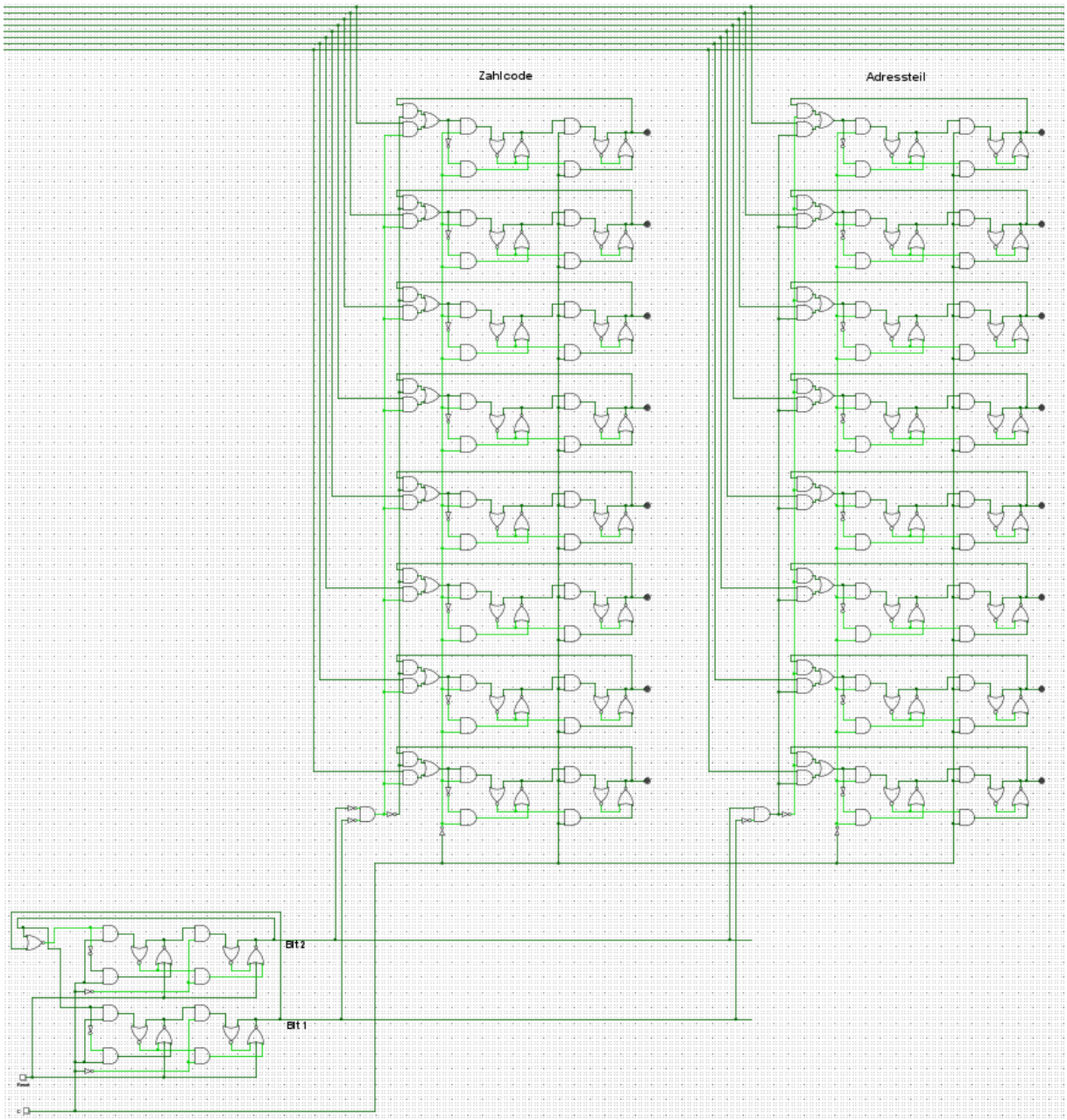
Diesen schaltet man nun vor die beiden D-Flipflops und gibt ihm als Eingabe deren

Ausgabe. Mit jeder fallenden Flanke des Taktes wechselt der Inhalt der Flipflops einen Schritt weiter, wie oben beschrieben:

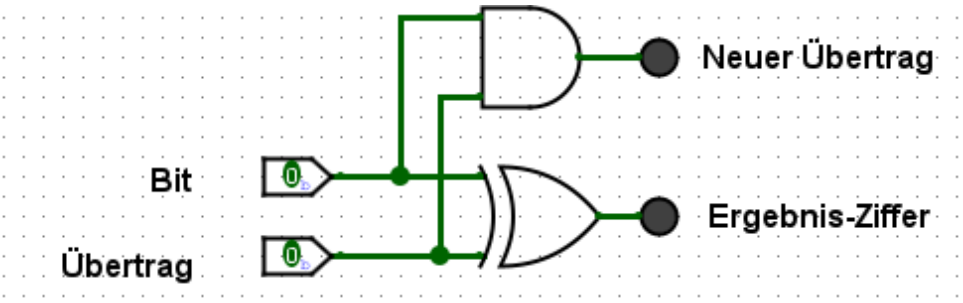


6.6.1 Register im Steuerwerk

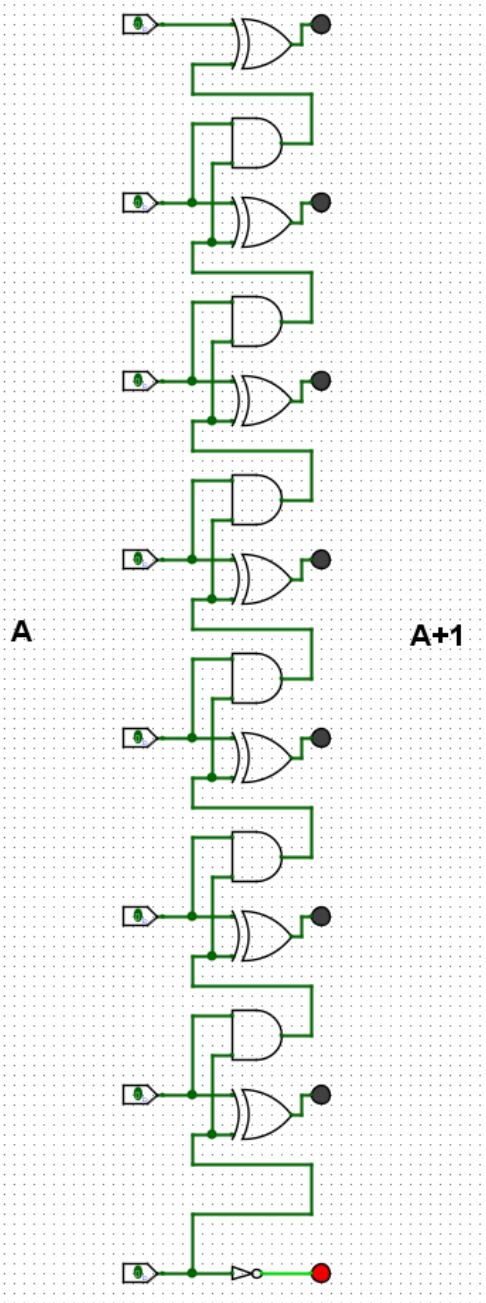
Jetzt muss nur noch der jeweilige Schritt ausgeführt werden. Dafür müssen allerdings noch drei weitere Register dem Steuerwerk hinzugefügt werden: zwei, in die der auszuführende Befehl geladen wird, in einen der Zahlcode und in den anderen der Adressteil, und eins, das die aktuelle Position im Programm speichert, anders formuliert die Speicheradresse, von der der nächste Befehl geladen wird. Dieses Register nennt man Befehlszähler.
Die beiden Befehlsregister werden an den Datenbus angeschlossen, wie auch die Register im Speicher. Ihr E-Eingang wird mit dem ersten Bit des Schrittzählers negiert verbunden. Da dieses nur an ist, bei Schritt 3 bzw. 10 und in den beiden vorherigen Schritten, in denen der Befehl geladen wird, aus ist. Damit allerdings zwischen diesen beiden Registern unterschieden wird, kombiniert man es noch durch ein AND-Gatter mit dem zweiten Bit, beim ersten Register in negierter Form. Diese Register aktualisieren sich bei der steigenden Flanke, da sie zusammen mit dem Speicher arbeiten:



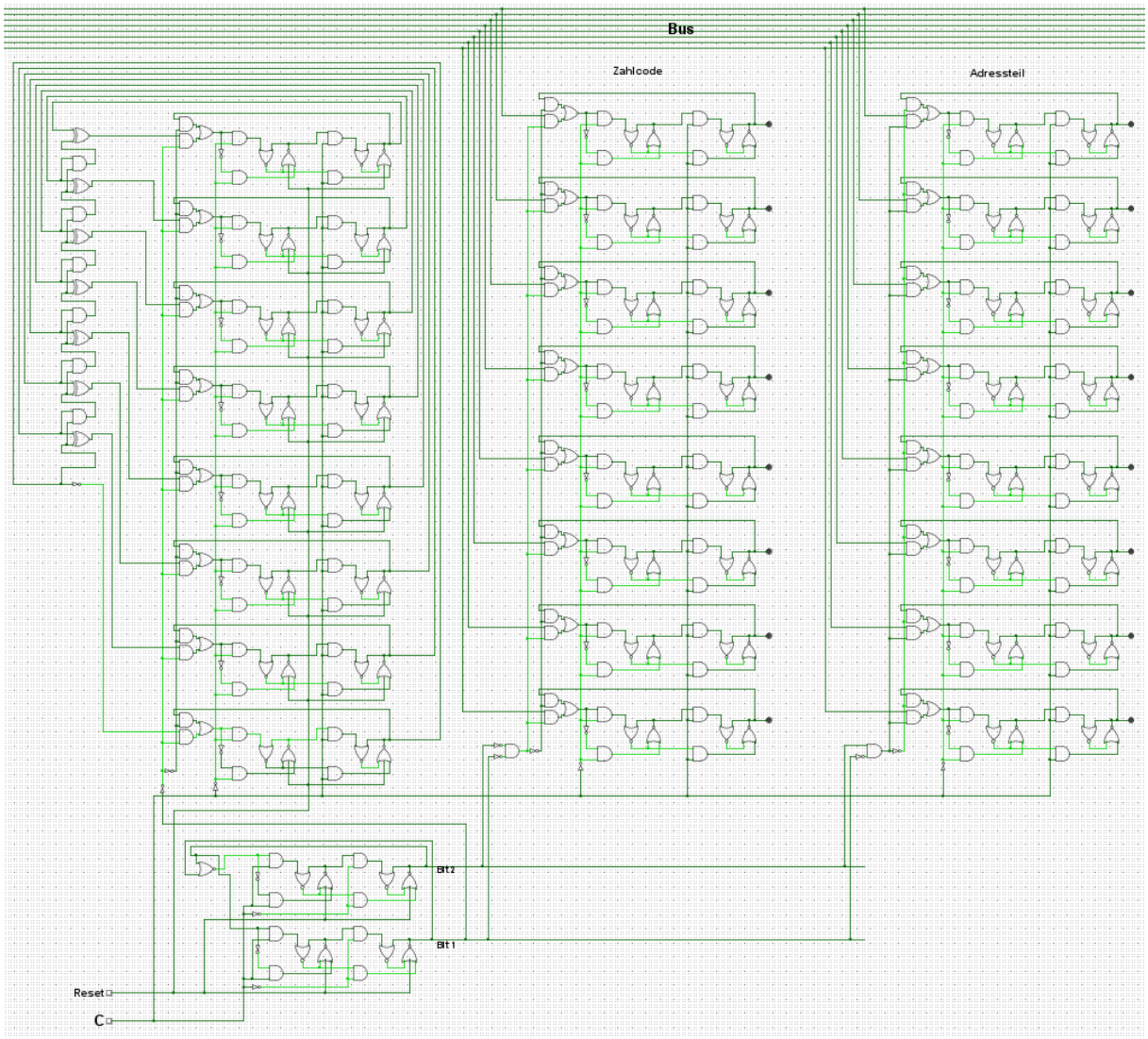
Ich habe dem Schrittzähler auch einen Reset-Eingang gegeben, der an allen internen SR-Flipflops am R Eingang angeschlossen ist, um diese alle zurückzusetzen. Dazu später mehr.
Der Befehlszähler ist ebenso ein Register. Er wird allerdings ähnlich zum Schrittzähler durch ein Schaltnetz aktualisiert. Dieses ist ein einfacher Addierer, der immer mit eins addiert, das entspricht einem Addierer, der nur eine Ziffer und Überträge rechnet. Damit ist ein Addierschritt viel einfacher, das Ergebnis Bit ist immer noch ein einfaches XOR, aber jetzt mit nur dem Übertrag und der Ziffer als Eingabe. Der neue Übertrag ist immer noch, ob mindestens zwei aktiviert sind, da es allerdings nur zwei gibt, entspricht das dem Schema eines einfachen AND-Gatters:



Diese schaltet man nun zusammen, wie bei einem normalen Addierer. Beim ersten Bit kann man allerdings, statt einen konstanten Übertrag von 1 zu übergeben, auch direkt die neue Ziffer durch ein NOT-Gatter und den neuen Übertrag als den Bit selber übergeben, da dies auf selbe hinausläuft. Der letzte Übertrag braucht auch nicht berechnet zu werden, da er sowieso keinen Effekt hat, da die Adressen gar nicht so groß sind:



Dieses Schaltnetz schaltet man nun vor den Befehlszähler, und dessen Ausgang wieder in diesen Addierer. Der E-Eingang dieses Registers ist an den negierten ersten Bit des Schrittzählers angeschlossen, damit nur bei Schritt 1 und Schritt 2 der Befehlszähler erhöht wird, also nur dann, wenn auch ein Befehl eingelesen wurde. Genauer gesagt passiert das sogar gleichzeitig. Es wird im selben Moment auf der ersten steigenden Flanke der Zahlcode des Befehls geladen und der Befehlszähler schon um einen erhöht, um den Adressteil zu laden. Genauso wird der Befehlszähler auch schon erhöht, während der Adressteil geladen wird:

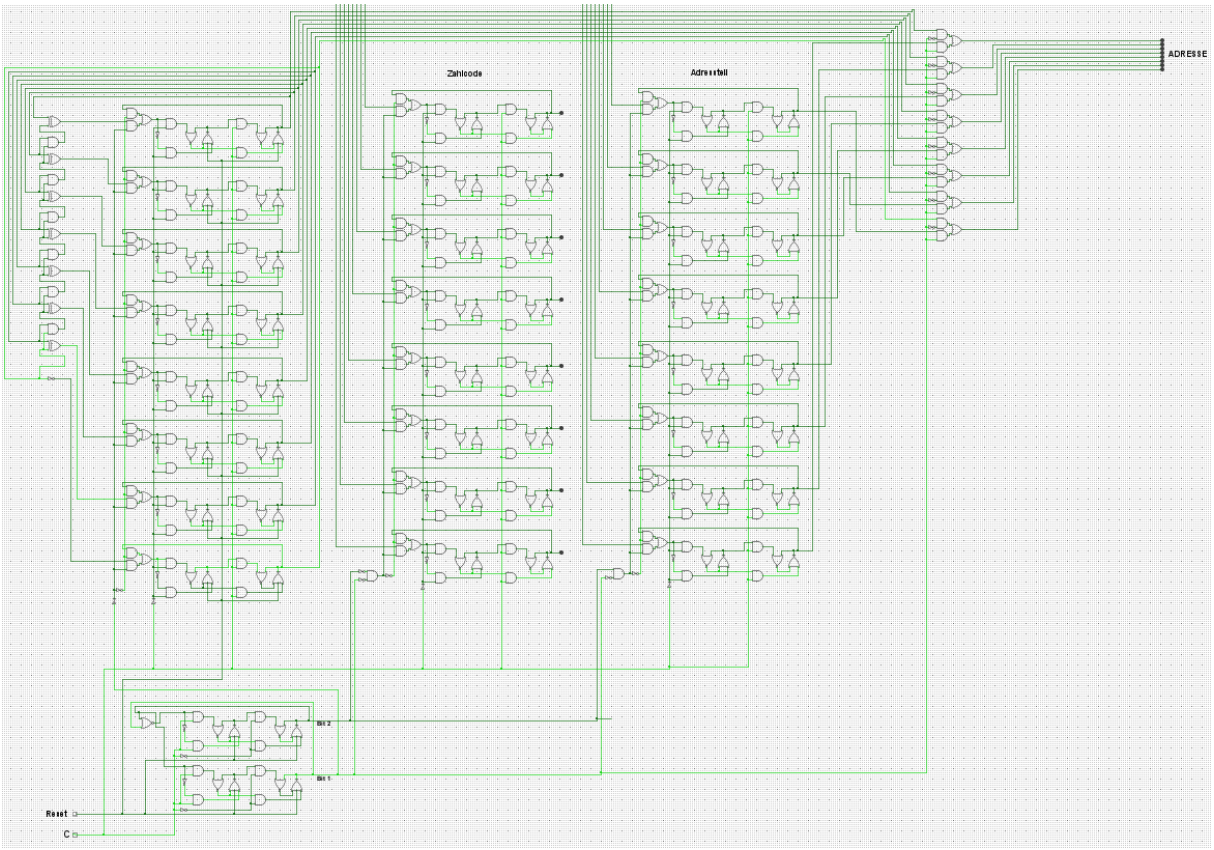


Auch den Befehlszähler habe ich an die Reset-Eingabe angeschlossen. Dies hat keine weiteren Gründe, außer dass es das Bedienen des Steuerwerks vereinfacht. Dadurch kann man immer zum Beginn eines Programmes, am Anfang des Speichers - Adresse 0 -, zurückspringen.

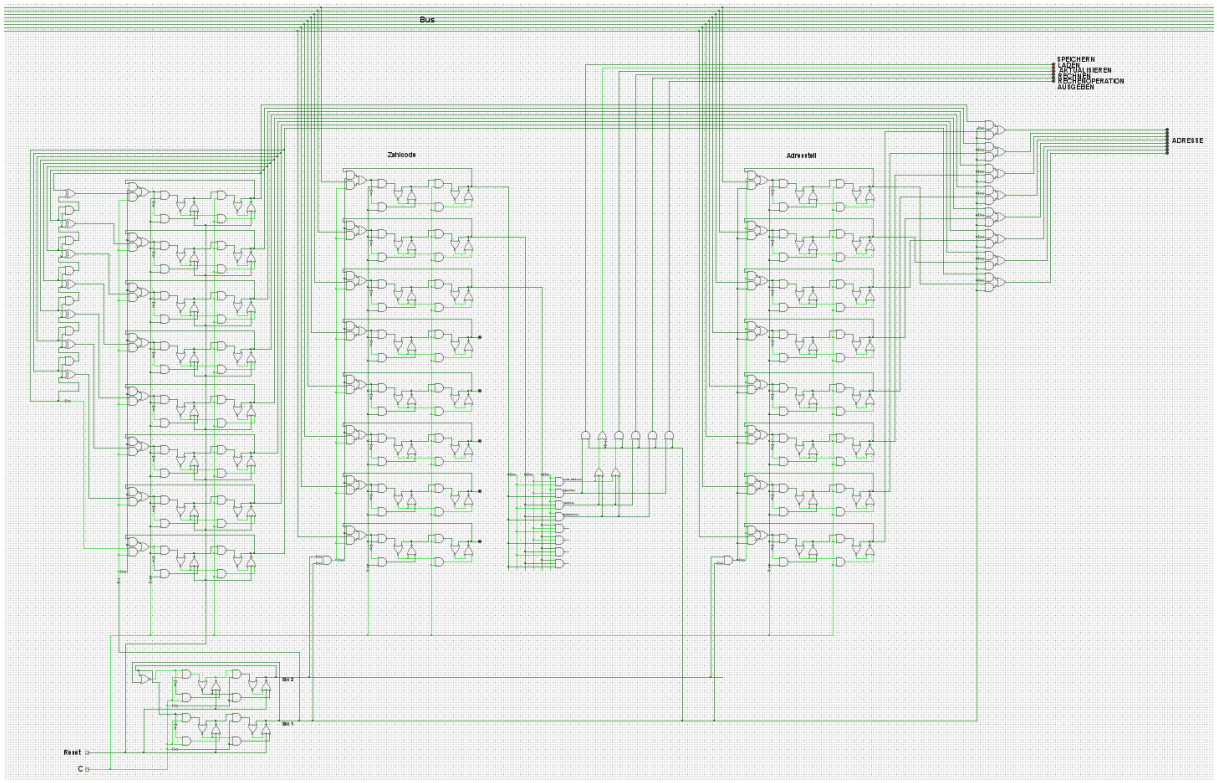
6.6.2 Einbau in das bestehende Steuerwerk

Um das Ganze funktional zu machen, muss man es auch an die Steuersignale anschließen. Falls gerade ein Befehl ausgeführt wird, also Bit 1 des Schrittzählers 1 ist, werden die Steu-

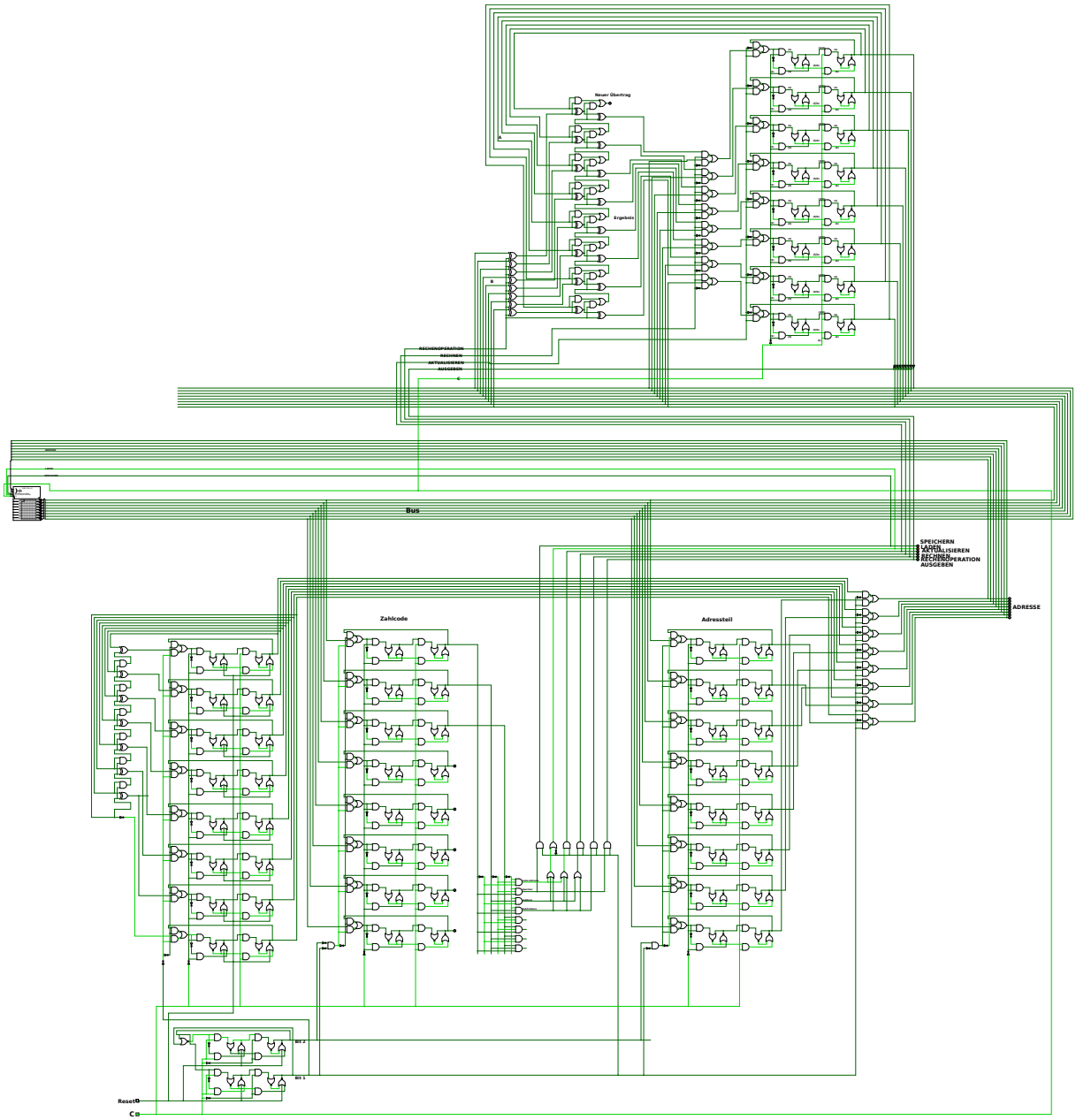
ersignale wie oben schon implementiert gesendet. Falls das nicht der Fall sein sollte, also gerade Befehlsteile aus dem Speicher geladen werden, muss nur das LADEN-Steuersignal gesendet werden und über das ADRESSE-Signal der Stand des Befehlszählers übermittelt werden; der Rest der Steuersignale bleibt aus.
Um das ADRESSE-Signal zwischen dem Befehlszähler und dem Adressteil zu wechseln, benötigt man eine Auswahlschaltung, wie sie schon für das Rechenwerk zum Laden vom Bus gebaut wurde. Diese muss nun abhängig vom ersten Bit des Schrittzählers den Adressteil von dem Register oder aus dem Befehlszähler laden. Diese Verkabelung kann z.B. wie folgt aussehen:



Das LADEN-Steuersignal muss, falls ein Befehl geladen wird, immer an sein, und sonst je nach Befehl mit dem oben erklärten Schaltnetz für die verschiedenen Befehle geschaltet werden. Das kann man mit einem OR-Gatter umsetzen. In dieses führt man einerseits das normale LADEN-Signal und einmal das verneinte erste Bit, da das OR-Gatter, wenn der zweite Eingang aus ist, die Eingabe des ersten Gatters unberührt wieder ausgibt, wenn es an ist, allerdings auch immer an ausgibt. Der verneinte erste Bit ist äquivalent zu der Information, ob gerade ein Befehl geladen wird, also wird immer 1 ausgegeben, wenn gerade ein Befehl geladen wird.
Analog dazu verbindet man die restlichen Steuersignale durch ein AND-Gatter. In die zweite Eingabe führt man das erste Bit, also die Information, ob gerade ein Befehl ausgeführt wird. Denn wenn das der Fall ist, wird das Signal übertragen und sonst 0 ausgegeben. Um dies zu implementieren, baut man nun das vorher gebaute Schaltnetz zu den Registern:



Das Steuerwerk kann man in dieser Form schon in den Computer einbauen. Hierfür schließt man alle Komponenten an denselben Bus an, in diesem Fall sind dies nur Speicherwerk und Rechenwerk. Man muss dann an das Takt-Signal nur noch einen Taktgeber anschließen und dieses auch an alle anderen Komponenten leiten. Außerdem muss man alle Steuersignale verbinden:



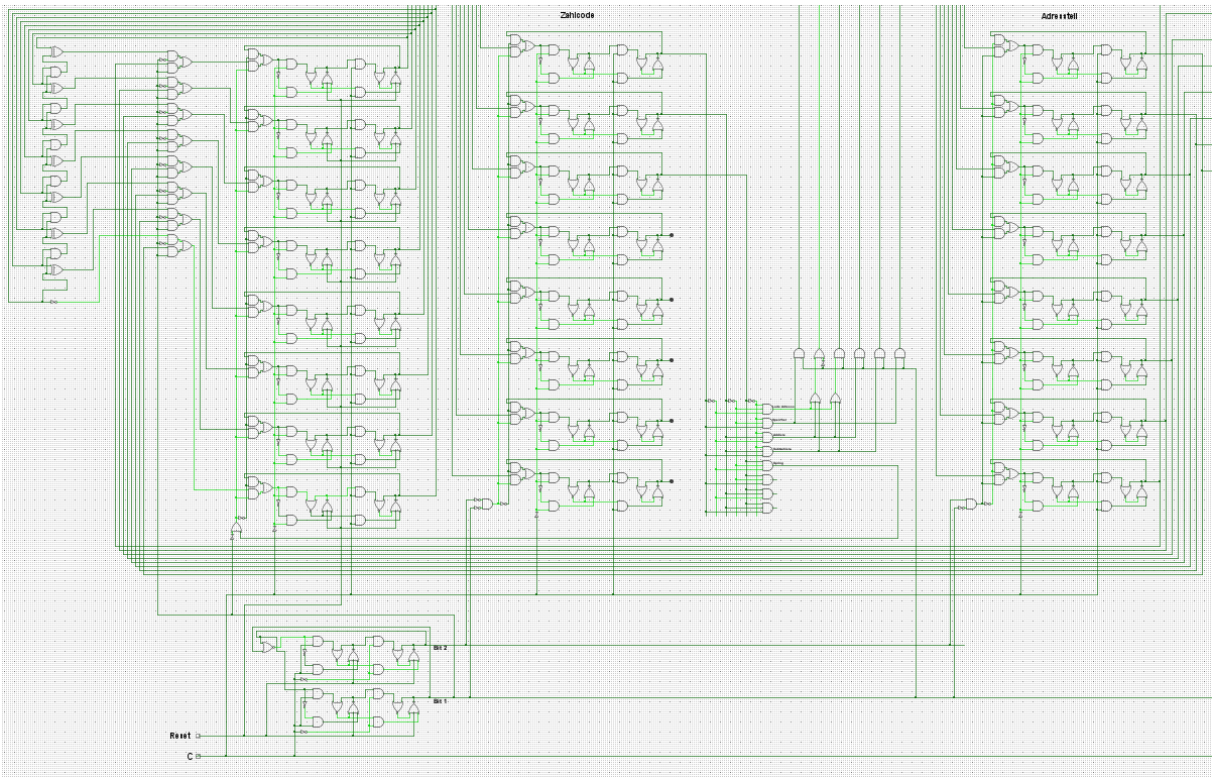
6.6.3 Sprungbefehle

Die Programme, die der Computer nun ausführen kann, können nun schon vieles berechnen. Allerdings bieten sie noch keine Funktion für Schleifen oder Bedingungen. Um dies zu implementieren, braucht man Sprungbefehle. Diese Befehle setzen den Befehlszähler auf eine neue Position, von der anschließend der nächste Befehl geladen wird. Zu den bisherigen Befehlen kommen nun 4 Sprungbefehle hinzu:

Befehl	Zahlcode	Parameter	Funktion
Lade	0	Adresse, aus der geladen werden soll	Lädt den Wert in den Akkumulator, der in der Speicherzelle hinterlegt ist
Speicher	1	Adresse, in die gespeichert werden soll	Speichert den Wert des Akkumulators in den Speicherwerk
Addiere	2	Adresse, dessen Inhalt addiert werden soll	Addiert Wert aus dem Speicherwerk auf den Akkumulator
Subtrahiere	3	Adresse, dessen Inhalt subtrahiert werden soll	Subtrahiert Wert aus dem Speicherwerk vom Akkumulator
Spring	4	Adresse, zu der gesprungen werden soll	Springt zur angegebenen Adresse
Spring falls 0	5	Adresse, zu der gesprungen werden soll	Springt, falls der Wert des Akkumulators 0 ist
Spring falls >0	6	Adresse, zu der gesprungen werden soll	Springt, falls der Wert des Akkumulators größer 0 ist (in der Zweierkomplementdarstellung)
Spring falls <0	7	Adresse, zu der gesprungen werden soll	Springt, falls der Wert des Akkumulators kleiner als 0 ist (in der Zweierkomplementdarstellung)

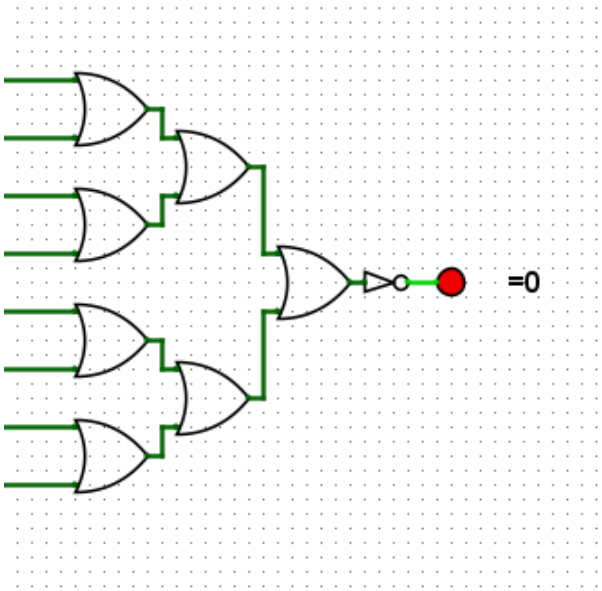
6.6.3.1 Bedingungsloser Sprungbefehl

Ein Sprungbefehl muss den Befehlszähler auf die im Adressteil des Befehls gegebene Adresse setzen, damit von dieser der nächste Befehl geladen wird, also das Programm so zusagen dorthin gesprungen ist. Um dies zu implementieren, muss man ermöglichen, dass der Befehlszähler nicht nur von dem Addierer, sondern auch von dem Adressteil geladen werden kann.
Dies kann man mithilfe einer Auswahlschaltung umsetzen, wie sie auch schon für das ADDRESSE-Signal benutzt wurde. Diese wählt danach aus, ob ein Befehl ausgeführt wird (Bit 1 Schrittzähler), ob vom Adressteil oder vom Addierer geladen wird. Nur mit diesem Aufbau allein geschieht allerdings noch nichts, denn das Befehlszähler-Register kann noch nicht laden, während ein Befehl ausgeführt wird, da der Aktivierungseingang (E) nur an ist, während ein Befehl aus dem Speicher geladen wird. Damit dennoch ein Sprungbefehl ausgeführt werden kann, muss man mit einem OR-Gatter diesen Eingang auch dann aktivieren, wenn ein Sprungbefehl ausgeführt wird. In dem momentanen einfachen Szenario eines immer springenden Sprungbefehls wird somit das dekodierte Signal für den Sprungbefehl am Befehlsdekodierer zu diesem OR-Gatter geleitet:

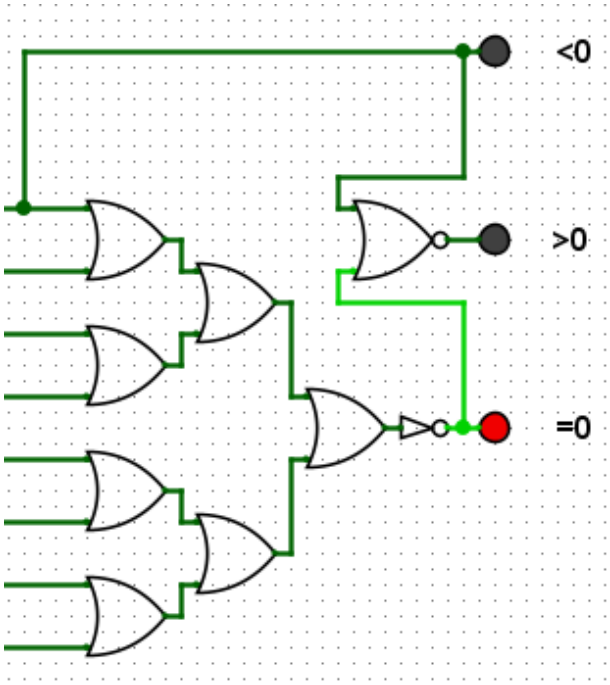


6.6.3.2 Bedingungen

Um den Sprungbefehlen Bedingungen zu geben, müssen erst an einer Stelle, die Bedingungen überprüft werden. Dies ist Aufgabe des Rechenwerks. Deshalb müssen dort Schaltungen gebaut werden, um die drei Bedingungen zu überprüfen, ob der Akkumulator null ist, ob er größer als null ist und ob er kleiner als null ist.
Die Überprüfung, ob der Akkumulator 0 ist, ist sehr einfach zu implementieren. Null in einer 8-Bit-Binärdarstellung ist nämlich 00000000. Also ist die Bedingung, die man überprüfen muss nur, ob keine der Stellen 1 bzw. an ist. Um dies umzusetzen, kann man erst die nicht verneinte Bedingung bauen und sie anschließend verneinen. Also überprüft man zuerst, ob irgendeines der Bits an ist, was einer einfachen OR-Schaltung entspricht, und verneint es dann:

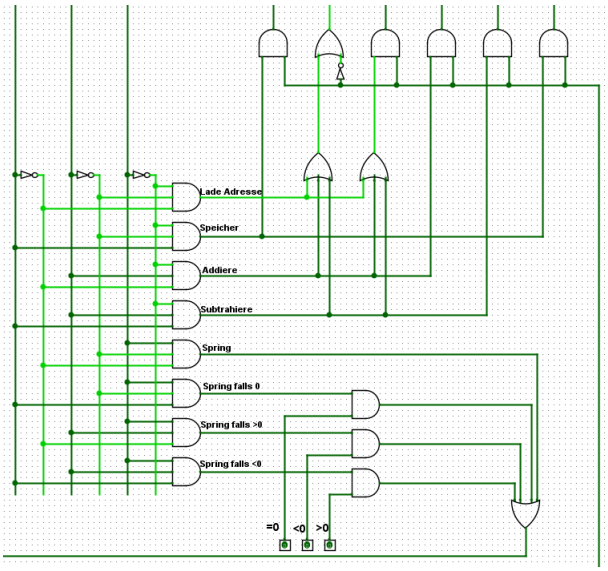


Die Überprüfung, ob der Akkumulator kleiner als 0 ist, ist sogar noch einfacher. Denn, da in der Zweierkomplementdarstellung das erste Bit einen Stellenwert von -128 hat und alle anderen zusammenaddiert nur auf 127 kommen, muss eine Zahl immer negativ bzw. kleiner als 0 sein. Also ist diese Überprüfung einfach nur das erste Bit des Akkumulators. Daraus ergibt sich auch automatisch die Überprüfung, ob die Zahl größer als 0 ist, denn, wenn sie weder kleiner noch gleich ist, muss sie größer sein. Das kann man durch ein NOR-Gatter implementieren:

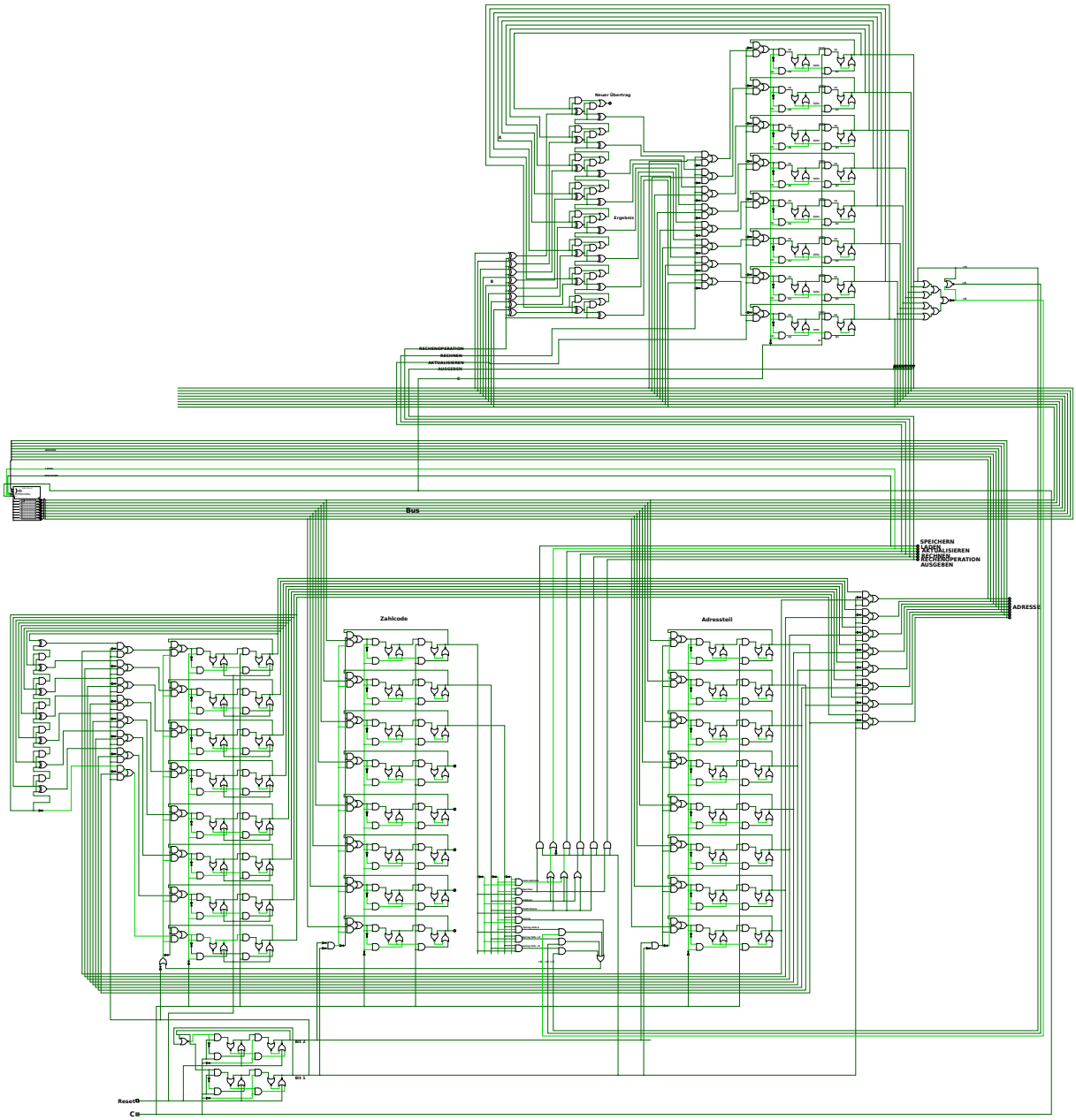


6.6.3.3 Bedingte Sprungbefehle

Nun, wo man Signale hat, die angeben, ob der Akkumulator größer, gleich oder kleiner ist, kann man die restlichen drei Sprungbefehle implementieren, indem man zur Ausführung des jeweiligen Befehls dann das Neuladen des Befehlszählers aktiviert, wenn dieser Befehl ausgewählt ist und die Bedingung erfüllt ist. Dies lässt sich dann für jeden bedingten Sprungbefehl durch ein AND-Gatter implementieren und diese Signale der vier Sprungbefehle durch ein OR-Gatter zusammenschalten:



Diese neue Version der Befehlsverarbeitung und die Erweiterung des Rechenwerks können nun in den Computer eingebaut werden. Anschließend verbindet man die neuen Ausgänge des Rechenwerks mit den neuen Eingängen des Steuerwerks. Damit hat man einen funktionierenden Computer, wenn auch einen sehr kleinen, der sehr wenige Funktionen und Optimierungen hat:


Neuer Übertrag
Ergebnis
RECHENOPERATION
BEGINNEN
AKTUALISIEREN
AUSGEBEN
C
Bus
SPEICHERN
LADEN
AKTUALISIEREN
RECHNEN
RECHENOPERATION
AUSGEBEN
ADRESSE
Zahlcode
Adressteil
Bit 2
Bit 1
Reset
Clk

Kapitel 7

Beispiele für Programme

Mit diesem Computer kann man aber dennoch schon erste einfache Programme schreiben. Von diesen habe ich im Folgenden beispielhaft einige aufgeführt. Zur Darstellungsweise: Der Speicher in dem genutzten Programm ist in Hexadezimaldarstellung, daher werden im Folgenden die Zahlen im Programmtext auch in Hexadezimal geschrieben.

7.1 Verdopplung

Das folgende Programm soll den Wert in dem Speicherregister unter der Adresse 10 (Hexadezimal also in dezimal 16) verdoppeln:

```
0: Lade (Zahlcode: 0)
1: 10
2: Addiere (Zahlcode: 2)
3: 10
4: Speicher (Zahlcode: 1)
5: 10

6: Spring (Zahlcode: 4)
7: 6
```

Dieses Programm lädt den Wert in der Speicherregister 10 in den Akkumulator, addiert dann dazu den Wert aus Speicherregister 10 und speichert diesen errechneten Wert anschließend wieder in dem Speicherregister 10. Der anschließende Sprungbefehl springt auf sich selbst, ist also einfach eine Endlosschleife, damit das Programm nicht weiter rechnet. Dies kann man nun mit Übersetzung aller Befehlswörter in die jeweiligen Zahlcodes in den Speicher schreiben. In der folgenden Abbildung sind nur die ersten 32 Speicherzellen zu sehen, die anderen sind nicht wichtig, da sie leer, also 0 sind. Als Beispielwert für die zu verdoppelnde Zahl ist hier 56 (Hexadezimaldarstellung, Dez.: 86):

```
00  00 10 02 10  01 10 04 06  00 00 00 00  00 00 00 00
10  56 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00
```

Nachdem man dieses Programm ausgeführt hat, sieht der Speicherinhalt schließlich, wie erwartet, wie folgt aus:

```
00  00 10 02 10  01 10 04 06  00 00 00 00  00 00 00 00
10  ac 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00
```

7.2 Multiplizieren

Das folgende Programm soll zwei positive Zahlen, a und b, miteinander multiplizieren können. a ist die Zahl auf Adresse 11 und b die Zahl auf Adresse 12, das Ergebnis soll auf Adresse 13 geschrieben werden.

```
0: Lade (Zahlcode: 0)
1: 13
2: Addiere (Zahlcode: 2)
3: 12
4: Speicher (Zahlcode: 1)
5: 13
6: Lade (Zahlcode: 0)
7: 11
8: Subtrahiere (Zahlcode: 3)
9: 10
a: Speicher (Zahlcode: 1)
b: 11
c: Spring falls >0 (Zahlcode: 6)
d: 0

e: Spring (Zahlcode: 4)
f: e

10: 1
```

Dieses Programm geht dadurch vor, dass es zuerst b auf das vorübergehende Ergebnis addiert, danach anschließend 1 von a subtrahiert - 1 ist als Konstante dafür in Adresse 10 gespeichert - und falls dies noch größer als null sein sollte, diesen ganzen Vorgang wiederholt. Es führt also eine Iteration aus, wobei a der Zähler ist. Hier wird das Programm auch auf gleiche Weise gestoppt.
Im Simulationsprogramm sieht das dann so aus - Beispielwerte $a = 7; b = 9$:

```
00  00 13 02 12  01 13 00 11  03 10 01 11  06 00 04 0e
10  01 07 09 00  00 00 00 00  00 00 00 00  00 00 00 00
```

Das Programm errechnet hier korrekterweise:

```
00  00 13 02 12  01 13 00 11  03 10 01 11  06 00 04 0e
10  01 00 09 3f  00 00 00 00  00 00 00 00  00 00 00 00
```

Als Kollateraleffekt wird a auch auf 0 gesetzt, da auf dieser Speicherzelle heruntergezählt wird. Wenn man das vermeiden wollte, müsste man a vorher auf eine andere Adresse kopieren und von dieser herunterzählen. Das kann man leicht umsetzen, indem man a in den Akkumulator lädt und schließlich an anderer Stelle wieder speichert.

7.3 Zeiger

Eine essenzielle Funktion moderner Programmiersprachen sind sogenannte Zeiger beziehungsweise Referenzen. Das sind Verweise auf bestimmte Stellen im Speicher. Also an sich nur Adressen. Allerdings sind diese selber im Speicher abgelegt und von diesen nun zu laden ist etwas komplizierter. Es gibt keinen Befehl, mit dem man dies direkt ausführen kann.
Man muss nun, um das irgendwie umzusetzen, in den Adressteil eines Lade-Befehls diese Adresse des Zeigers bekommen. Da das Register des Adressteils nur vom Speicher im Prozess des Befehls-Laden gefüllt wird, ist die einzige Möglichkeit, dort den Zeiger zu laden, dass dieser vorher unter der von dem Befehlsladeprozess geladen werdende Adresse gespeichert wird.
Dieses Programm führt als Beispiel für einen solchen Zeiger folgendes durch: Es ersetzt den Inhalt unter Adresse 10 mit dem Inhalt der Adresse, die dort unter Adresse 10 hinterlegt ist:

```
0: Lade (Zahlcode: 0)
1: 10
2: Speicher (Zahlcode: 1)
3: 5
4: Lade (Zahlcode: 0)
5: 0
6: Speicher (Zahlcode: 1)
7: 10

8: Spring (Zahlcode: 4)
9: 8
```

Hier wird Besagtes umgesetzt. Es wird die Adresse aus Speicherregister 10 geladen und in dem später aufgerufenen Befehl im Adressteil gespeichert. Dann wird dieser Befehl mit der vorher eingetragenen Adresse ausgeführt, die als Zeiger hinterlegt war. Der damit geladene Wert wird dann unter der Adresse 10 gespeichert.
In der Simulation sieht das schließlich so aus (als Beispiel ist hier unter der Adresse 10 die Adresse 29 hinterlegt und unter dieser der Wert 42):

```
00  00 10 01 05  00 00 01 10  04 08 00 00  00 00 00 00
10  29 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00
20  00 00 00 00  00 00 00 00  00 42 00 00  00 00 00 00
```

Wie erwartet, sieht das Ergebnis wie folgt aus:

```
00  00 10 01 05  00 29 01 10  04 08 00 00  00 00 00 00
10  42 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00
20  00 00 00 00  00 00 00 00  00 42 00 00  00 00 00 00
```

7.4 Die Fibonacci-Folge

Die Fibonacci-Folge ist eine der berühmtesten Folgen natürlicher Zahlen. Fibonacci erfand sie, um die Population von Kaninchen zu beschreiben. Bei dieser Folge ist jede Zahl die Summe der beiden vorherigen. Die Reihe fängt mit 1 und 1 an.[1]
Das folgende Programm soll die zehn weiteren Fibonacci-Zahlen berechnen, in den Adressen 20 - 2c:

```
 0: Lade (Zahlcode: 0)
 1: 1e
 2: Speicher (Zahlcode: 1)
 3: f
 4: Addiere (Zahlcode: 2)
 5: 1c
 6: Speicher (Zahlcode: 1)
 7: 1e
 8: Speicher (Zahlcode: 1)
 9: 11
 a: Addiere (Zahlcode: 2)
 b: 1c
 c: Speicher (Zahlcode: 1)
 d: 13

 e: Lade (Zahlcode: 0)
 f: 0
10: Addiere (Zahlcode: 2)
11: 0
12: Speicher (Zahlcode: 1)
13: 0

14: Lade (Zahlcode: 0)
15: 1e
16: Subtrahiere (Zahlcode: 3)
17: 1d
18: Spring falls <0 (Zahlcode: 7)
19: 0

1a: Spring (Zahlcode: 4)
1b: 1a

1c: 1
1d: 2a
1e: 20

20: 1
21: 1
```

In diesem Programm wird mit einem Zeiger, i, gearbeitet. Dieser ist in 1e gespeichert und

[1] Jüstel, *Fibonacci-Folge*.

hat den Startwert 20, verweist also auf die erste Fibonacci-Zahl. Die Zeilen 0 - d tragen ihn teilweise in abgeänderter Form in die nachfolgenden Befehle ein. Zuerst wird i geladen. So wird i auch in den Lade-Befehl in Zeile f, beziehungsweise e+f, eingetragen. Dann wird der Akkumulator um 1 erhöht - er wird addiert mit der Konstante 1, die im Speicher unter der Adresse 1c hinterlegt ist. So wird der Zeiger, i, anschließend wieder gespeichert, sodass er beim nächsten Mal um eins erhöht ist. In dieser um 1 erhöhten Form wird er auch im Addiere-Befehl in Zeile 11 gespeichert. Daraufhin wird er um noch eins erhöht, in dem Speicher-Befehl in Zeile 13 gespeichert, sodass in den Zeilen e-13 effektiv Folgendes steht:

```
 e: Lade
 f: i
10: Addiere
11: i + 1
12: Speicher
13: i + 2
```

Diese Zeilen sorgen damit dafür, dass in dem Speicherregister $i+2$ die Summe der Zahlen aus den Speicherregistern i und $i+1$ gespeichert werden, also die nächste Fibonacci-Zahl errechnet wird.
Die darauf folgenden Zeilen 14 - 19 sorgen für die Schleifenlogik - i wurde bereits am Anfang erhöht - hier muss nun validiert werden, ob i noch kleiner als 2a ist. Denn solange dies der Fall ist, soll der Vorgang wiederholt werden. Da es insgesamt zehnmal wiederholt werden muss, um zehn weitere Fibonacci Zahlen zu berechnen und i bei 20 startet, sind die Werte von i in den zehn Schleifendurchläufen 20-29. Ab $i \geq 2a$ wird daher nicht weiter wiederholt. Um nun zu prüfen, ob $i < 2a$ wird zuerst die Differenz von i und 2a genommen, da $i < 2a \Leftrightarrow i - 2a < 0$. Dies wird dann anschließend durch den Spring-falls-$<$ 0-Befehl überprüft. Falls nicht gesprungen wird, geht das Programm zum Stoppbefehl, von dem aus es nicht mehr weiter läuft.
Trägt man das in den Speicher in der Simulation ein, sieht das wie folgt aus:

```
00 00 1e 01 0f  02 1c 01 1e  01 11 02 1c  01 13 00 00
10 02 00 01 00  00 1e 03 1d  07 00 04 1a  01 2a 20 00
20 01 01 00 00  00 00 00 00  00 00 00 00  00 00 00 00
```

Das Programm funktioniert und erzeugt erfolgreich zehn weitere Fibonacci-Zahlen:

```
00 00 1e 01 0f  02 1c 01 1e  01 11 02 1c  01 13 00 29
10 02 2a 01 2b  00 1e 03 1d  07 00 04 1a  01 2a 2a 00
20 01 01 02 03  05 08 0d 15  22 37 59 90  00 00 00 00
```

7.5 Euklidischer Algorithmus

Der euklidische Algorithmus ist ein Algorithmus zur Bestimmung des größten gemeinsamen Teilers zweier Zahlen. Er wird, wenn auch in erweiterter Form, auch heute noch in modernen Computern eingesetzt, z.B. für die RSA-Verschlüsselung. In einfacher Form lässt er sich durch folgenden Pseudocode ausdrücken:[2]

[2]Klinger, "Euklidischer Algorithmus".

```
EUKLID(a, b):
    WENN a = 0 DANN
        GIB ZURUECK b
    SONST
        SOLANGE b != 0
            WENN a > b DANN
                a = a - b
            SONST
                b = b - a
        GIB ZURUECK a
```

Diesen kann man nun Schritt für Schritt in einen für den Computer ausführbaren Programmtext umwandeln. a und b sind unter Adresse 30 und 31 gespeichert, das Ergebnis soll unter der Adresse 32 gespeichert werden:

```
 0: Lade (Zahlcode: 0)
 1: 30
 2: Spring falls 0 (Zahlcode: 5)
 3: 1e
 4: Lade (Zahlcode: 0)
 5: 31
 6: Spring falls 0 (Zahlcode: 5)
 7: 18
 8: Subtrahiere (Zahlcode: 3)
 9: 30
 a: Spring falls <0 (Zahlcode: 7)
 b: 10
 c: Speicher (Zahlcode: 1)
 d: 31
 e: Spring (Zahlcode: 4)
 f: 4
10: Lade (Zahlcode: 0)
11: 30
12: Subtrahiere (Zahlcode: 3)
13: 31
14: Speicher (Zahlcode: 1)
15: 30
16: Spring (Zahlcode: 4)
17: 4
18: Lade (Zahlcode: 0)
19: 30
1a: Speicher (Zahlcode: 1)
1b: 32
1c: Spring (Zahlcode: 4)
1d: 1c
1e: Lade (Zahlcode: 0)
1f: 31
20: Speicher (Zahlcode: 1)
21: 32
22: Spring (Zahlcode: 4)
```

23: 22

Die ersten Zeilen 0-3 überprüfen die erste Bedingung, ob $a = 0$. Wenn das der Fall ist, ist b das Ergebnis. Das Programm springt daher zu Zeile 1e und speichert b als Ergebnis. Dann kommt dort schon der Stoppbefehl.
Dann kommt die Schleife. Um $b! = 0$ zu validieren, wird b in den Akkumulator geladen. Dann wird, falls b 0 ist, die Schleife übersprungen. Und auf analoge Weise, wie vorhin b, a als Ergebnis zurückgegeben.
Dann beginnt mit Zeile 8 der innere Teil der Schleife. Von dem sich immer noch im Akkumulator befindenden b wird nun a abgezogen, sodass dieser nun $b - a$ enthält, mit dieser Differenz wird nun validiert, ob $a > b$. Da $a > b \Leftrightarrow a - b > 0 \Leftrightarrow b - a < 0$ kann dies mit einem Spring-falls-$<$ 0-Befehl validiert werden. Wenn nicht gesprungen wird, also nicht $a > b$ muss nach Pseudocode b auf $b - a$ gesetzt werden, da das praktischerweise immer noch im Akkumulator ist, muss es nur unter der Adresse von b (31) gespeichert werden. Anschließend daran wird zurück zum Beginn der Schleife gesprungen.
Falls allerdings $a > b$ das Programm also zu Zeile 10 gesprungen wird, muss a auf $a - b$ gesetzt werden. Um $a - b$ zu berechnen wird a in den Akkumulator geladen und b subtrahiert. Das Ergebnis wird anschließend unter der Adresse von a gespeichert. Auch hier wird danach wieder zum Beginn der Schleife gesprungen.
Damit ist der euklidische Algorithmus vollständig implementiert (in einfacher Form). In der Simulation kann man nun testen, ob er funktioniert - Beispielwerte 44 und 12 beziehungsweise 2c und c:

00	00	30	05	1e	00	31	05	18	03	30	07	10	01	31	04	04
10	00	30	03	31	01	30	04	04	00	30	01	32	04	1c	00	31
20	01	32	04	22	00	00	00	00	00	00	00	00	00	00	00	00
30	2c	0c	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Wie man sieht, funktioniert das Programm und hat den korrekten größten gemeinsamen Teiler 4 gefunden:

00	00	30	05	1e	00	31	05	18	03	30	07	10	01	31	04	04
10	00	30	03	31	01	30	04	04	00	30	01	32	04	1c	00	31
20	01	32	04	22	00	00	00	00	00	00	00	00	00	00	00	00
30	04	00	04	00	00	00	00	00	00	00	00	00	00	00	00	00

Kapitel 8

Fazit

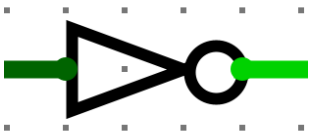
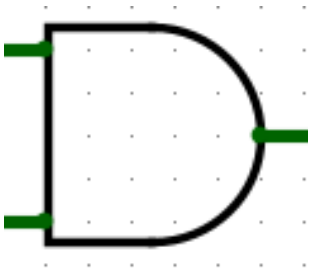
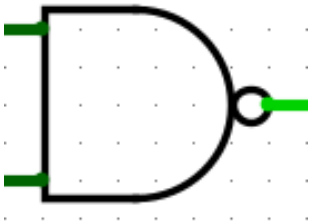
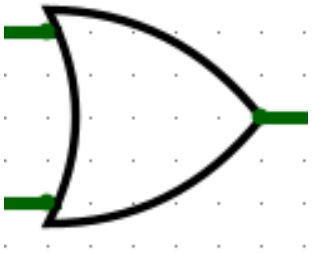
Während ich bei der Erarbeitung der von-Neumann Architektur noch sehr eng an den Quellen gearbeitet habe und dies auch bei dem Speicherwerk anfangs noch getan habe, habe ich beim Bau des Rechenwerks sehr frei gearbeitet und mir die Schaltnetze selber hergeleitet, teilweise mit Vorwissen, das ich nicht mehr zu genauen Quellen zuordnen kann. Beim Steuerwerk habe ich kaum Quellen für die praktische Implementation gefunden; alle von mir gelesenen Quellen blieben immer sehr abstrakt, was das Steuerwerk angeht. Das Steuerwerk habe ich, samt aller Komponenten außer den Registern, selbst gestaltet. Dadurch habe ich einen noch tieferen Einblick in die Funktionsweise des Steuerwerks bekommen.
Schließlich habe ich das höchste angestrebte Ziel geschafft und eigene Programme für den Computer geschrieben. Allerdings liegt das auch daran, dass ich doch weniger Zeit benötigt habe, als ursprünglich geplant, da ich das Ganze aus Kostengründen nicht praktisch umsetzen konnte. Es wäre allerdings unter Umständen möglich, die Kosten zu reduzieren, indem man mehr vorgefertigte Komponenten verwendet. So würde man z.B. das Speicherwerk als einen vorgefertigten Chip einbauen, ähnlich wie es schon in der Simulation gelöst ist. Das hatte ich sowieso vor, das sorgt allerdings grade mal dafür, dass die Kosten in den dreistelligen Bereich fallen. Man müsste also z.B. auch die Register im Rechenwerk und Steuerwerk mit vorgefertigten Komponenten ersetzen und ebenso die dort verbauten Auswahlschaltungen etc. Da dies aber dem angestrebten grundlegenden Einblick entgegenstehen würde, habe ich mich in dieser Projektarbeit dagegen entschieden.
Andere Möglichkeiten, dieses Projekt fortzusetzen, wäre die Implementation moderner Computermechaniken wie z.B. Zwischenspeicher im Steuerwerk, sogenannten Cache. Eine solche Mechanik wäre aber auch das Aufteilen der Befehlsverarbeitung in mehr Takte, da man die Taktfrequenz dann erhöhen kann. Das erhöht zwar noch nicht die Geschwindigkeit, in der die Befehle verarbeitet werden. Doch durch die Implementation sogenannter CPU-Pipelines kann man die Geschwindigkeit erhöhen, indem die Befehle wie am Förderband durch die vielen Schritte gehen. So braucht ein einzelner Befehl zwar nicht weniger Zeit, um ausgeführt zu werden, es werden aber dennoch mehr Befehle pro Zeit ausgeführt.
Eine einfachere Möglichkeit, das Projekt zu erweitern, wäre das Hinzufügen von Ein- und Ausgabe, man könnte, wenn man vier Knöpfe und einen kleinen Bildschirm anschließt, sogar auf dem Computer Snake spielen.
Es gibt bei diesem Thema fast unbegrenzt Möglichkeiten, das Projekt noch zu erweitern. In der Arbeit habe ich z.B. auch schon das Rechnen mit Zahlen größer als 127 bzw. kleiner als -128 angesprochen, die man dann auf zwei Speicherwörter aufteilen müssten. Auch

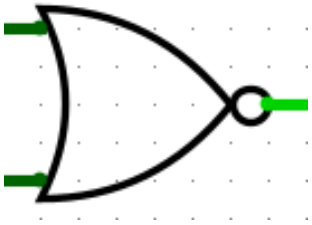
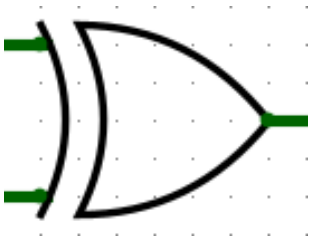
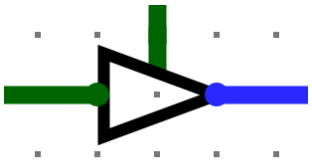
könnte man dem Rechenwerk noch Rechenoperationen hinzufügen wie z.B. das Multiplizieren oder eventuell auch Dividieren.
Analog zu diesem Projekt, welches einen Computer mit der von-Neumann-Architektur gebaut hat, könnte man auch einen Computer mit einer anderen Architektur bauen. Eine ebenfalls öfters genutzte Architektur ist die Harvard-Architektur. Mit dieser könnte man auch einen Computer entwerfen.

Insgesamt habe ich aber mit diesem Projekt schon mein Verständnis der Mechaniken eines Computers sehr weit vertieft, besonders der Aufbau des Steuerwerks ist mir jetzt deutlich eingängiger. Im Gesamten bin ich sehr zufrieden mit dem Projekt, da ich als Bestätigung der Funktionalität sogar selbst geschriebene Programme auf dem Computer ausführen konnte, auch wenn es nur in der Simulation geschehen ist.

Kapitel 9

Legende

NOT(-Gate/Gatter)		negiert den Eingabewert: aus 0 wird 1 und aus 1 wird 0
AND(-Gate/Gatter)		führt eine logische UND-Operation aus: nur 1, wenn auch beide Eingänge 1 sind
NAND(-Gate/Gatter)		negiertes AND: nur wenn beide Eingänge 1 sind, wird die Ausgabe 0
OR(-Gate/Gatter)		führt eine logische ODER-Operation aus: sobald ein Eingang 1 ist, ist auch die Ausgabe 1

NOR(-Gate/Gatter)		negiertes OR: nur wenn mindestens ein Eingang 1 ist, ist auch die Ausgabe 0
XOR(-Gate/Gatter)		exklusives ODER: nur wenn genau ein Eingang 1 ist, ist auch die Ausgabe 1
Tristate-Puffer/-Buffer		übermittelt den eingegebenen Zustand nur, wenn die Aktivierungseingabe eingeschaltet ist, mehr dazu im Unterkapitel zu Bussen und der Tristate-Logik

Literatur

Kelling: Aufbau, Konzepte und Funktionsweisen von Computersystemen **von-neumann**

Carsten Kelling. "Aufbau, Konzepte und Funktionsweisen von Computersystemen". Abgerufen am 15. 5. 2024. Diplomarbeit. Universität Hamburg, 1996. URL: https://tams.informatik.uni-hamburg.de/applets/baukasten/DA/VNR_Einleitung.html.

Finley: Two's Complement **tc**

Thomas Finley. "Two's Complement". Englisch. In: (Apr. 2000). Abgerufen am 28. 5. 2024. Cornell University. URL: https://www.cs.cornell.edu/~tomf/notes/cps104/twoscomp.html.

Thormählen: Sequentielle Schaltungen **flipflops**

Thorsten Thormählen. "Sequentielle Schaltungen". Abgerufen am 2. 6. 2024; Seite 11. Philipps Universität Marburg. Dezember 2021. URL: https://www.mathematik.uni-marburg.de/~thormae/lectures/ti1/ti_7_3_ger_web.html#1.

Git-Hub: Logisim-evolution **logisim**

siehe Git-Hub. *Logisim-evolution*. 2022. URL: https://github.com/logisim-evolution/logisim-evolution (besucht am 05. 06. 2024).

Franz: Rechnerarchitektur **harvard**

Dr.-Ing. Elke Franz. "Rechnerarchitektur". Deutsch. Presentation. Abgerufen am 2. 6. 2024; Seite 11. Technische Universität Dresden. 2017/2018. URL: http://dud.inf.tu-dresden.de/~ef1/informatik/vkinf1_03_1s.pdf.

Jüstel: Fibonacci-Folge **fib**

Prof. Dr. T. Jüstel. *Fibonacci-Folge*. Abgerufen am 8. 6. 2024. FH Münster. Erscheinungsdatum unbekannt. URL: https://www.fh-muenster.de/ciw/downloads/personal/juestel/juestel/Fibonacci-Folge.pdf.

Klinger: Euklidischer Algorithmus **euclid**

Dr. Marcel Klinger. "Euklidischer Algorithmus". Abgerufen am 8. 6. 2024. 2020/21. URL: https://www.klinger.nrw/wp-content/uploads/2021/01/107_Euklidischer-Algorithmus.pdf.